



INTEGRA
SENAI

REGRAFIK

SENAI Serviço Nacional
de Aprendizagem
Industrial

SUMÁRIO

1 Título

1.1 EQUIPE

1.2 PROBLEMA

1.3 SOLUÇÃO

1.3.1 ÁREA TECNOLÓGICA DA SOLUÇÃO..... 7

1.3.2 JUSTIFICATIVA..... 7

1.3.3 OBJETIVOS 7

1.3.4 DESENVOLVIMENTO 7

1.3.5 VIABILIDADE TÉCNICA 7

1.3.6 VIABILIDADE ECONÔMICA 7

1.3.7 RESULTADOS E CONCLUSÃO..... 7

1.4 ANEXO

1.4.1 BMG CANVAS..... 7

1.4.2 SITUAÇÃO DE APRENDIZAGEM..... 7

1 REGRAPHIK — SISTEMA DE GESTÃO DE ESTOQUE REVERSO

1.1 EQUIPE

Unidade SENAI: Senai Afonso Greco – Nova Lima

Instrutor orientador: Frederico Martins Aguiar

ALUNO	CURSO	FUNÇÃO NO PROJETO
Bruno Maia	Técnico em Desenvolvimento de Sistemas	Documentação técnica, Modelagem do banco de dados (Conceitual, Lógico e Físico)
Otávio Henrique	Técnico em Desenvolvimento de Sistemas	Desenvolvimento do cliente desktop (WPF / MVVM)
Lucas Aquino	Técnico em Desenvolvimento de Sistemas	Desenvolvimento do cliente desktop (WPF / MVVM) e Github
Luna Beatriz	Técnico em Desenvolvimento de Sistemas	Desenvolvimento da API REST (ASP.NET Core), Desenvolvimento front-end
Kaio Alves	Técnico em Desenvolvimento de Sistemas	Integração com Firebase e Google Maps Places API

Observação: as funções acima foram estimadas a partir dos artefatos técnicos do projeto e devem ser validadas/ajustadas pela equipe.

1.2 PROBLEMA

O setor gráfico é um dos segmentos industriais que mais gera resíduos sólidos no Brasil. Empresas de médio e grande porte produzem diariamente toneladas de aparas de papel, retalhos de vinil, restos de cartão, lona e PVC que, na ausência de processos estruturados de gestão, são simplesmente descartados. Esse comportamento acarreta custos financeiros com o descarte, impacto ambiental mensurável e, sobretudo, desperdício de matéria-prima que poderia ser reintegrada à cadeia produtiva.

A ausência de ferramentas digitais específicas para o controle de resíduos no setor gráfico obriga as empresas a recorrerem a planilhas manuais ou a simplesmente ignorar o potencial reaproveitável de seus descartes. Essa demanda foi identificada de forma concreta na empresa AML, onde o gerenciamento inadequado de resíduos representa perda econômica estimada e impacto ambiental evitável.

1.3 SOLUÇÃO

O ReGraphik propõe um ciclo completo de gestão de estoque reverso: desde o cadastro e classificação dos resíduos gerados, passando pela sugestão de rotas de reaproveitamento, até a localização de pontos de coleta na região — digitalizando um processo hoje realizado de forma manual ou inexistente.

1.3.1 ÁREA TECNOLÓGICA DA SOLUÇÃO

Desenvolvimento de Sistemas — a solução é composta por uma API REST em ASP.NET Core responsável pelas regras de negócio e persistência de dados, um cliente desktop em WPF (.NET) estruturado no padrão MVVM, um banco de dados em nuvem NoSQL (Firebase Realtime Database) e integração com a Google Maps Places API para geolocalização de pontos de coleta.

1.3.2 JUSTIFICATIVA

O projeto ReGraphik nasce da necessidade real identificada na empresa AML, onde o gerenciamento inadequado de resíduos representa perda econômica estimada e impacto ambiental evitável. A digitalização deste processo, combinada com sugestões automatizadas de reaproveitamento e integração com serviços de geolocalização, representa um avanço concreto em direção à economia circular no segmento gráfico.

1.3.3 OBJETIVOS

1.3.3.1 *Objetivo Geral*

Desenvolver um sistema de gestão de estoque reverso para o setor gráfico, composto por uma API REST em ASP.NET Core e um cliente desktop WPF, estruturado no padrão MVVM, com integração a serviços externos de banco de dados em nuvem e geolocalização.

1.3.3.2 *Objetivos Específicos*

- Modelar e implementar o banco de dados relacional com cinco entidades (CadastroUsuarios, TipoMaterial, CadastroResiduos, Sugestoes e SugestoesResiduos);
- Desenvolver uma API REST documentada (Swagger) com endpoints CRUD para todas as entidades do domínio;
- Implementar o padrão MVVM no cliente WPF, com BaseViewModel, RelayCommand e bindings declarativos;
- Integrar a Google Maps Places API para localização de pontos de coleta georreferenciados;
- Persistir os dados no Firebase Realtime Database, garantindo sincronização em nuvem;
- Documentar o sistema conforme os artefatos técnicos exigidos: Mini Mundo, Modelo Conceitual, Modelo Lógico e Modelo Físico.

1.3.4 DESENVOLVIMENTO

Esta seção detalha o processo de construção do ReGraphik: a metodologia adotada, o contexto de negócio (Mini Mundo), a modelagem de dados, a arquitetura do sistema, a API REST, os módulos funcionais, o processo de desenvolvimento, os requisitos, os casos de uso, o histórico evolutivo e as regras de negócio implementadas.

1.3.4.1 *Metodologia*

O projeto foi desenvolvido de forma iterativa e incremental, com divisão em três fases: configuração do ambiente e modelagem inicial (Fase 1), desenvolvimento da interface e integração com mapas (Fase 2) e construção da API REST e integração completa (Fase 3). O controle de versão foi realizado via Git/GitHub, com commits registrados ao longo de todo o desenvolvimento.

1.3.4.2 Mini Mundo da Demanda

Este tópico descreve o contexto organizacional que motivou o desenvolvimento do ReGraphik, os atores envolvidos e as regras de negócio que orientaram a modelagem do sistema.

Contexto Organizacional

Para sanar os gargalos da empresa AML, o fluxo operacional do Mini Mundo do sistema ReGraphik foi padronizado sob uma arquitetura de microserviços e tabelas normalizadas. O ciclo de vida inicia-se com o mapeamento e validação relacional das entidades. O Administrador do sistema pré-configura a tabela de TipoMaterial. Quando o operador executa o descarte fabril, o sistema captura suas credenciais a partir da tabela CadastroUsuarios e cria uma instância na tabela CadastroResiduos, associando-a obrigatoriamente às chaves estrangeiras correspondentes. O motor de busca realiza o cruzamento com as tabelas de Sugestoes por meio da entidade associativa SugestoesResiduos, registrando a data exata da aplicação da economia circular. Caso o material seja classificado como sem reuso interno, a API intercepta a localização geográfica, busca pontos externos via mapas, valida a redundância dos dados existentes e atualiza o inventário geral em tempo real na nuvem Firebase.

Atores do Sistema

Empresa (AML ou outras gráficas):

- Cadastra usuários no sistema;
- Utiliza o sistema como ferramenta de gestão operacional.

Usuário do Sistema:

- Realiza login com credenciais cadastradas;
- Cadastra resíduos gerados no processo produtivo;
- Consulta o estoque reverso atualizado;
- Aplica sugestões de reaproveitamento aos resíduos cadastrados;
- Gera relatórios operacionais.

Administrador:

- Gerencia usuários do sistema;
- Gerencia tipos de materiais cadastrados;

- Importa sugestões em lote via planilha.

Sistema ReGraphik:

- Controla o estoque reverso de resíduos;
- Sugere formas de reaproveitamento por tipo de material;
- Gera dashboards e relatórios gerenciais.

Regras de Negócio Gerais

- Um usuário pertence obrigatoriamente a uma empresa;
- Um resíduo deve ser classificado por um tipo de material;
- Um resíduo pode receber várias sugestões de reaproveitamento;
- Uma sugestão pode ser aplicada a múltiplos resíduos (relação N:N);
- O sistema registra automaticamente a data de cadastro de cada entidade;
- O sistema controla o status do resíduo com os estados: Em Estoque, Reaproveitado e Descartado;
- A exclusão de resíduos é restrita ao perfil Administrador e gera auditoria.

1.3.4.3 Modelagem do Banco de Dados

O sistema foi modelado em três níveis: conceitual, lógico e físico, conforme as metodologias de modelagem relacional adotadas no curso. O modelo conceitual segue a notação do BRModelo (Diagrama Entidade-Relacionamento), representando cinco entidades principais.

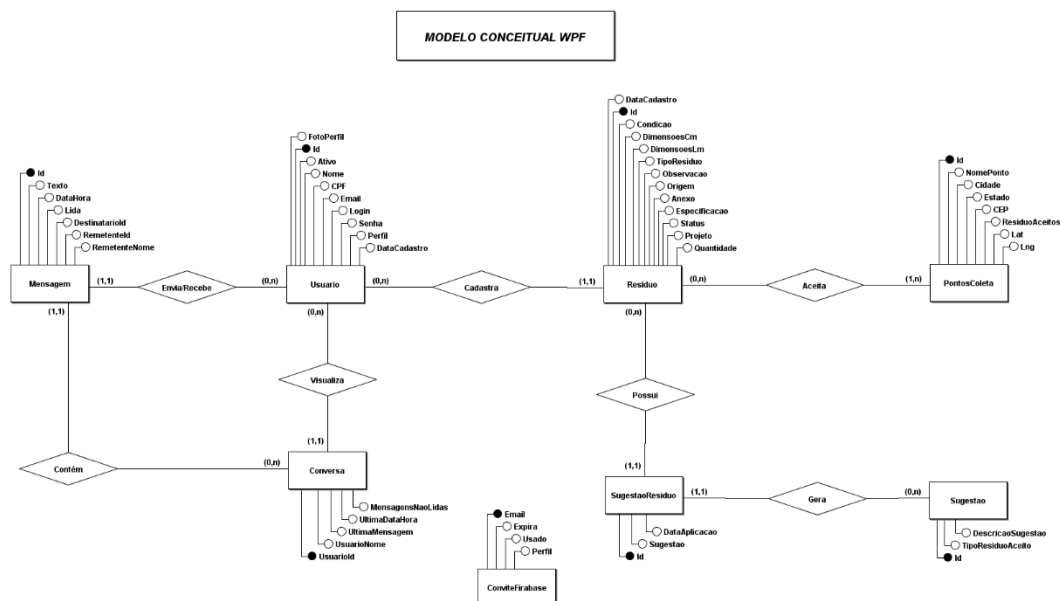


Figura 1. Modelo Relacional do banco de dados WPF.

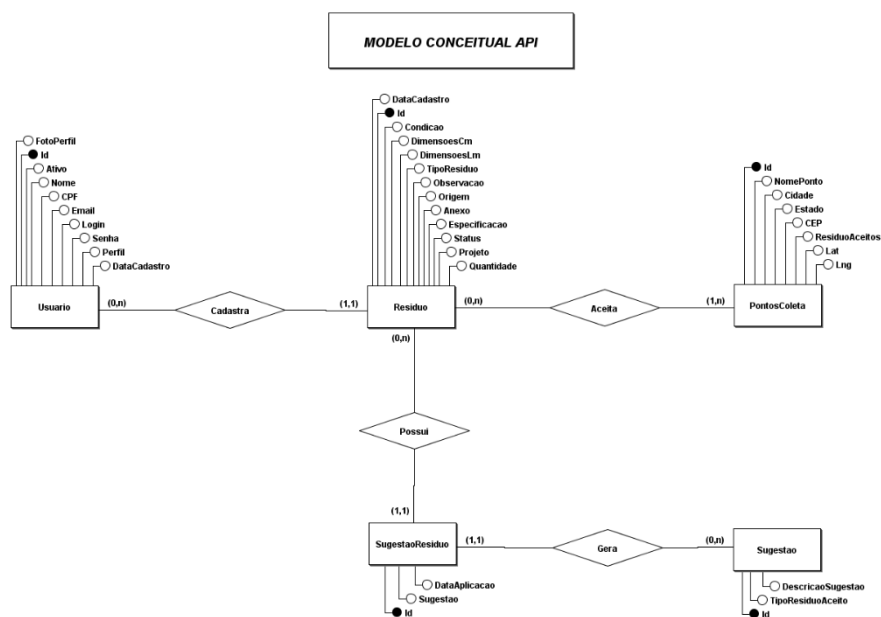


Figura 2. Modelo Relacional banco de dados API

Os relacionamentos definidos são: USUARIO 1:N CADASTRO_RESIDUOS (um usuário pode cadastrar vários resíduos); TIPO_MATERIAL 1:N CADASTRO_RESIDUOS e 1:N SUGESTOES; e CADASTRO_RESIDUOS N:N SUGESTOES, resolvido pela entidade associativa SUGESTOES_RESIDUOS.

No modelo lógico, as chaves primárias e estrangeiras (ID, IdUsuario, IdTipoMaterial) foram unificadas para aceitar tanto valores inteiros quanto string, evitando quebras de integridade referencial em chamadas assíncronas. As cinco tabelas relacionais resultantes — CadastroUsuarios, TipoMaterial, CadastroResiduos, Sugestoes e SugestoesResiduos — replicam a estrutura conceitual com tipos de dados definidos (VARCHAR, DATETIME, DECIMAL, ENUM, entre outros).

1.3.4.4 Arquitetura do Sistema

O ReGraphik adota uma arquitetura em camadas, separando claramente as responsabilidades entre o cliente desktop, a API REST e o banco de dados em nuvem. O sistema é composto por três camadas principais: Cliente WPF (Frontend Desktop, XAML/C# com padrão MVVM), API REST em ASP.NET Core (Backend) e Firebase Realtime Database (Persistência NoSQL em nuvem). Adicionalmente, integra a Google Maps Places API, consumida tanto pela API REST quanto pelo cliente desktop, que renderiza o mapa via Leaflet.js em um componente WebBrowser/WebView2.

Camada	Componentes	Responsabilidade
VIEW	MainWindow, DashboardPage, ResiduosPage, EstoqueReversoPage, MapaPage, RelatoriosPage (XAML)	Apresentação visual; bindings declarativos; sem lógica de negócio
VIEWMODEL	BaseViewModel, ResiduoViewModel, LoginViewModel, MapViewModel (C#)	Lógica de apresentação; INotifyPropertyChanged; ObservableCollections e RelayCommand
MODEL	Usuario, Residuo, Sugestao, SugestaoResiduo, PontosColeta (POCOs)	Entidades do domínio, sem dependência de UI

Camada	Componentes	Responsabilidade
SERVICE LAYER	GooglePlacesService, ApiService (C#)	Integração HTTP com API REST e Google Maps, injetada nos ViewModels

Todos os ViewModels herdam de BaseViewModel, que implementa INotifyPropertyChanged, permitindo que a View reaja automaticamente a mudanças de propriedades via Data Binding. O padrão Command é implementado pela classe RelayCommand, que encapsula delegates para execução e validação de comandos, eliminando o uso de event handlers no code-behind.

A estrutura de diretórios do repositório separa claramente o backend (ApiRestReGraphik — Controllers, Services, Models, Data), o frontend (ReGraphik — Views/Pages, ViewModels, Models, Services, Commands) e a pasta de Modelagem (PDFs dos modelos Conceitual, Lógico, Físico e MVVM).

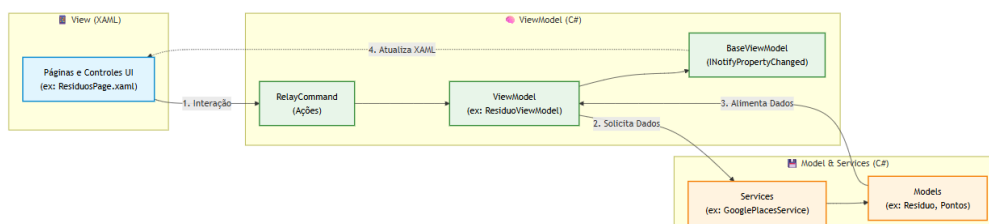


Figura — Fluxo de comunicação entre as camadas (View → ViewModel → Command → API REST → Firebase).

O fluxo de comunicação segue a sequência: WPF Client (View → ViewModel → Command) realiza chamadas HTTP/REST em JSON à API REST (Controllers → Services → Firebase SDK), que persiste os dados no Firebase Realtime Database via HTTPS e consulta a Google Maps Places API (NearbySearch/TextSearch) quando necessário.

1.3.4.5 API REST — Endpoints e Integração

A API REST do ReGraphik é desenvolvida em ASP.NET Core e exposta no domínio <https://webregraphik.runasp.net>. Ela é responsável por receber as requisições do cliente WPF, aplicar as regras de negócio e persistir os dados no Firebase Realtime Database. A API expõe 5 controllers com operações CRUD completas: Usuario, PontosColeta, Residuos, Sugestao e SugestaoResiduos.

Método	Endpoint	Descrição
GET	/api/residuos	Lista resíduos com filtros de status e tipo
GET	/api/residuos/{id}	Retorna resíduo com histórico de sugestões aplicadas
POST	/api/residuos	Cadastra novo resíduo — valida tipo, dimensões e usuário
PUT	/api/residuos/{id}/status	Atualiza status com log de alteração
DELETE	/api/residuos/{id}	Remove resíduo — requer perfil ADMIN, gera auditoria
GET	/api/usuarios	Lista usuários — restrito ao perfil ADMIN
POST	/api/sugestoes/aplicar	Aplica sugestão de reaproveitamento
GET	/api/coleta/proximos	Retorna pontos de coleta por lat/lng (Google Places)
POST	/api/PontosColeta/google?cidade=	Cadastra pontos via Google Maps Places API e salva no Firebase

Fluxo detalhado — busca de pontos de coleta: (1) o usuário clica em 'Buscar' na MapaPage; (2) o evento aciona o code-behind, que monta a URL da API com a cidade informada; (3) a API constrói a query para a Google Maps Places API; (4) o resultado é persistido no Firebase (pontos_coleta); (5) a API retorna a lista de pontos como JSON; (6) o cliente WPF gera o HTML com Leaflet.js e renderiza no componente WebBrowser.

A API é documentada via Swagger/OpenAPI (Swashbuckle), com interface interativa disponível em /swagger após a inicialização, permitindo explorar endpoints e testar chamadas diretamente no navegador. O endpoint /api/PontosColeta/google valida se a cidade já está

cadastrada no Firebase antes de consultar o Google Maps, evitando requisições externas redundantes.

1.3.4.6 Módulos do Sistema

O ReGraphik é organizado em módulos funcionais, cada um com escopo e status de desenvolvimento definidos:

Módulo	Descrição	Status
Mapa / Pontos de Coleta	Integração com Google Places API para localização de pontos de coleta próximos	Disponível
Dashboard	KPIs e indicadores de estoque reverso em tempo real	Em Construção
Cadastro de Resíduo	Triagem, classificação e registro de materiais com suporte a anexos	Em Construção
Sugestões	Motor de recomendação de reaproveitamento por tipo de material	Em Construção
Relatórios	Exportação de histórico e dados operacionais	Em Construção
Chat Interno	Canal de comunicação entre usuários da plataforma	Implementação Futura
Central de Notificações	Alertas automáticos de estoque e pendências	Implementação Futura
Log de Consulta	Auditoria e rastreabilidade de operações	Implementação Futura

1.3.4.7 Processo de Desenvolvimento

O desenvolvimento foi organizado em três fases iterativas, com escopo definido e entregáveis concretos, evidenciados pelo histórico de commits no repositório GitHub:

Fase	Atividades	Status
Fase 1 — Setup	Estrutura da solução C#, BaseViewModel, INotifyPropertyChanged, RelayCommand, modelagem inicial do banco	Concluído
Fase 2 — Core UI	WPF XAML (layout principal), tela de Login + ViewModel, Mapa de coleta + GooglePlacesService, início do CRUD de Resíduos	Concluído
Fase 3 — API REST	ApiRestReGraphik criado, Controllers (Resíduos, Usuários, Sugestões), integração API ↔ WPF Client	Em Progresso

O projeto utiliza Git para controle de versão, com repositório hospedado no GitHub (<https://github.com/BrunoMaiaSenai/ReGraphikApp>), demonstrando histórico consistente de desenvolvimento colaborativo. O ambiente de desenvolvimento adotado é o Visual Studio 2022 Community Edition, com SDK .NET 8/.NET 9, gerenciamento de pacotes via NuGet e npm, sistema operacional Windows 11 e banco de dados de desenvolvimento Firebase Realtime Database.

1.3.4.8 Requisitos do Sistema

Requisitos Funcionais

- RF-001 — Autenticação e Controle de Acesso: login com usuário e senha criptografada, diferenciando perfis comuns e Administradores.
- RF-002 — Cadastro e Triagem de Resíduos: registro de resíduos com especificações técnicas, origem, quantidade, dimensões, estado físico e anexos fotográficos.
- RF-003 — Atualização Automática de Status: gestão do estado do resíduo entre Em Estoque, Reaproveitado e Descartado.
- RF-004 — Motor de Sugestões de Reaproveitamento: correlação automática de sugestões com base no tipo de material.
- RF-005 — Geolocalização de Pontos de Coleta: busca de pontos de reciclagem por cidade, com dados georreferenciados em tempo real.

- RF-006 — Sincronização Automática com Firebase: persistência automática de novos pontos de coleta para consultas offline.
- RF-007 — Auditoria e Exclusão Segura: exclusão restrita a administradores, com trilha de auditoria.
- RF-008 — Geração de Relatórios e Dashboards: indicadores visuais (KPIs) e exportação de dados em formatos abertos (CSV).

Requisitos Não Funcionais

- RNF-001 — Desempenho e Concorrência: uso de programação assíncrona (async/await) para não bloquear a thread principal do cliente.
- RNF-002 — Compatibilidade de Ambiente: cliente WPF obrigatoriamente em Windows 10/11 (64 bits), com redistribuíveis do .NET 8 Runtime.
- RNF-003 — Disponibilidade do Banco de Dados: Firebase Realtime Database seguindo o SLA do plano contratado.
- RNF-004 — Escalabilidade Arquitetural: MVVM desacoplado, permitindo substituição de serviços via injeção de dependência.
- RNF-005 — Renderização de Mapas Leve: exibição cartográfica via Leaflet.js dentro do componente WebView2/WebBrowser.
- RNF-006 — Segurança da Informação: senhas armazenadas com algoritmo de hash, nunca em texto claro.

1.3.4.9 Casos de Uso

Diagrama de Caso de Uso — Login:

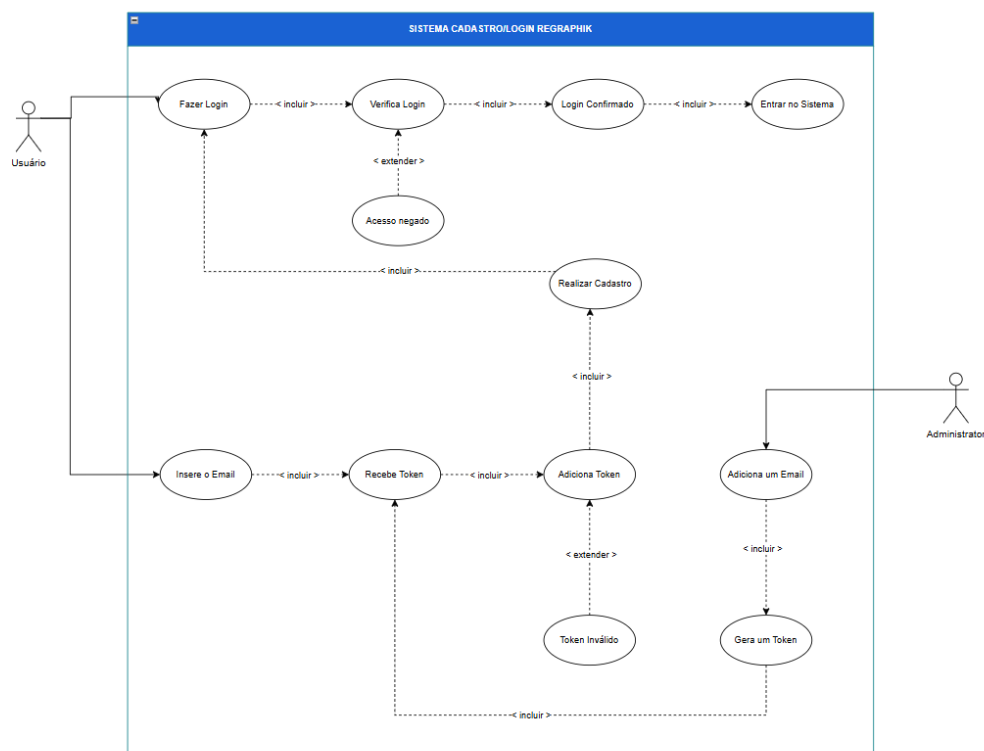


Diagrama de Caso de Uso — Cadastro de Resíduos:

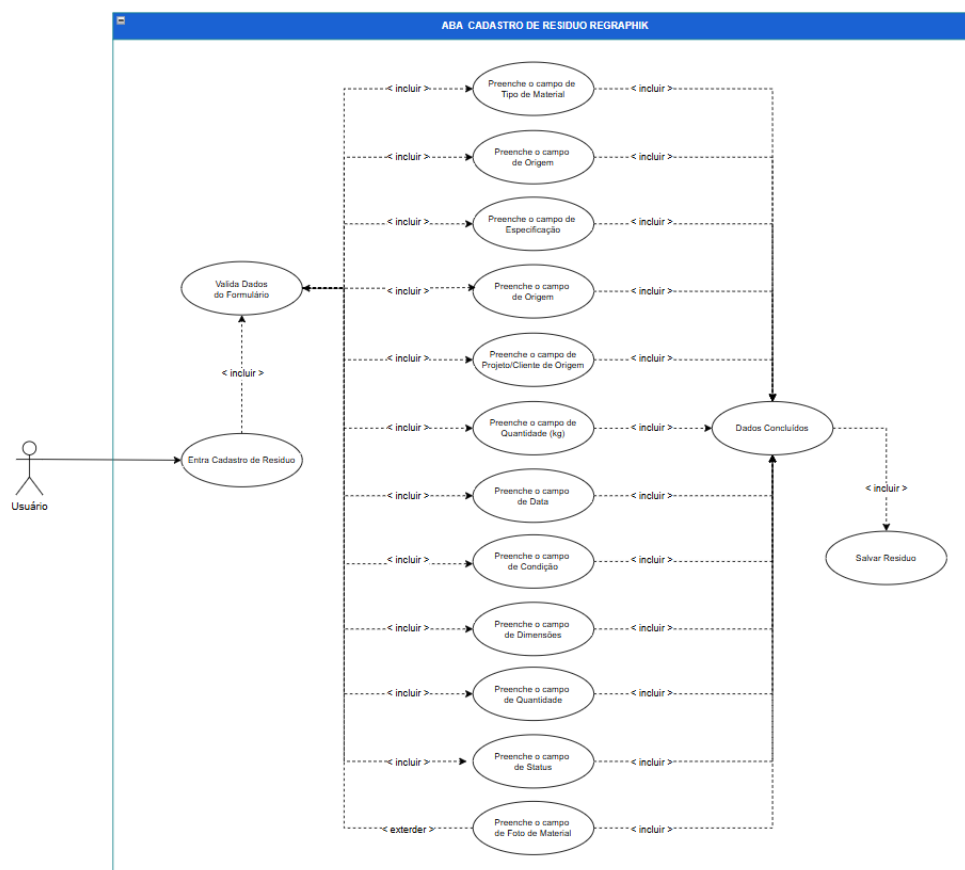
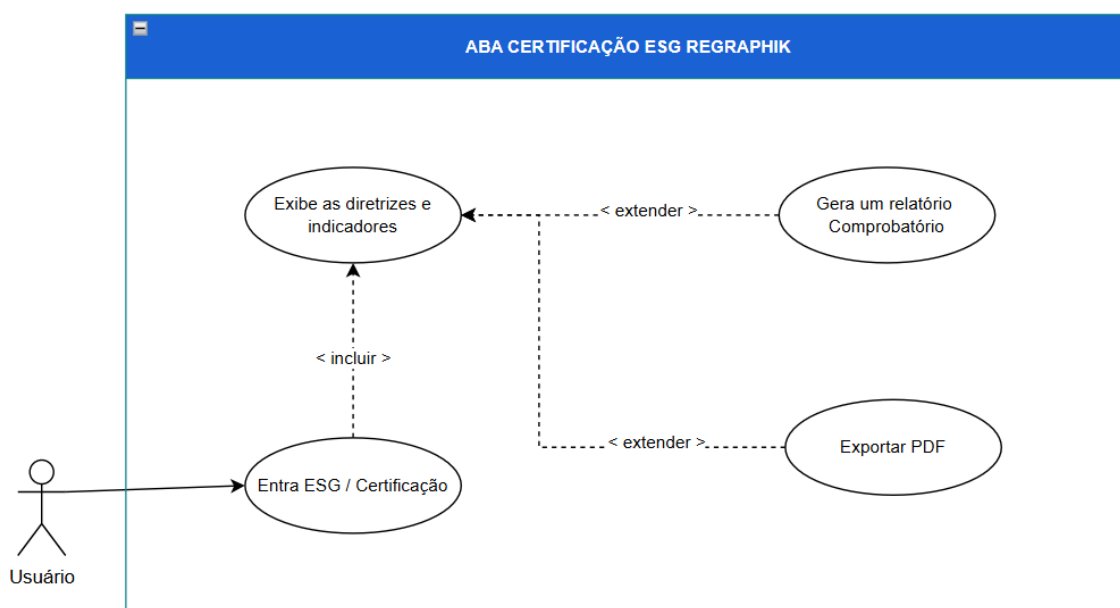
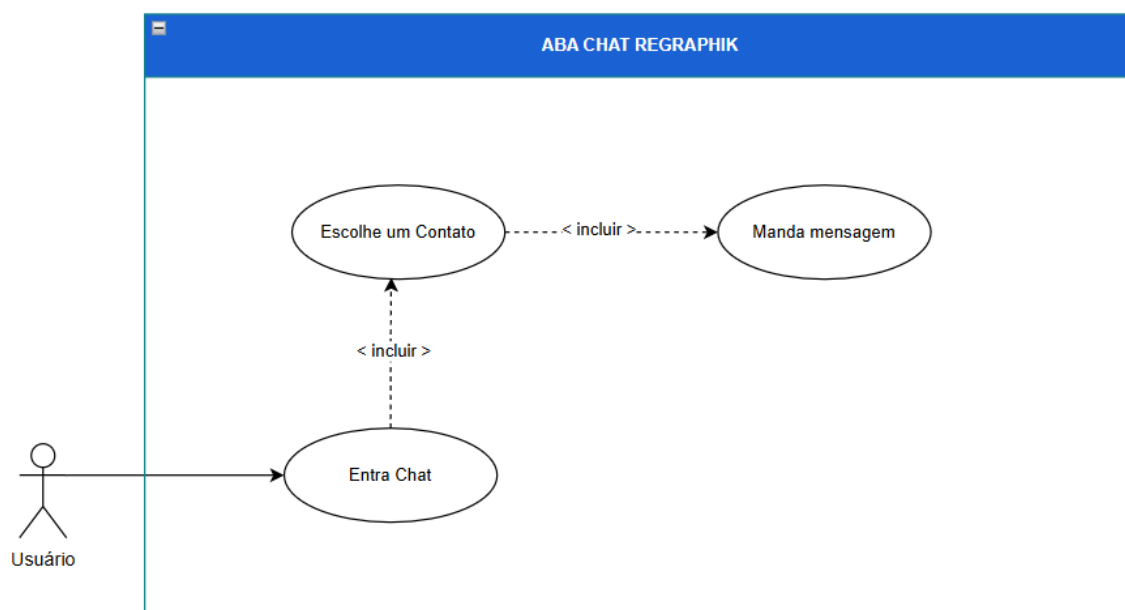


Diagrama de Caso de Uso — Certificação ESG:**Diagrama de Caso de Uso — Chat:****Diagrama de Caso de Uso — Dashboard:**

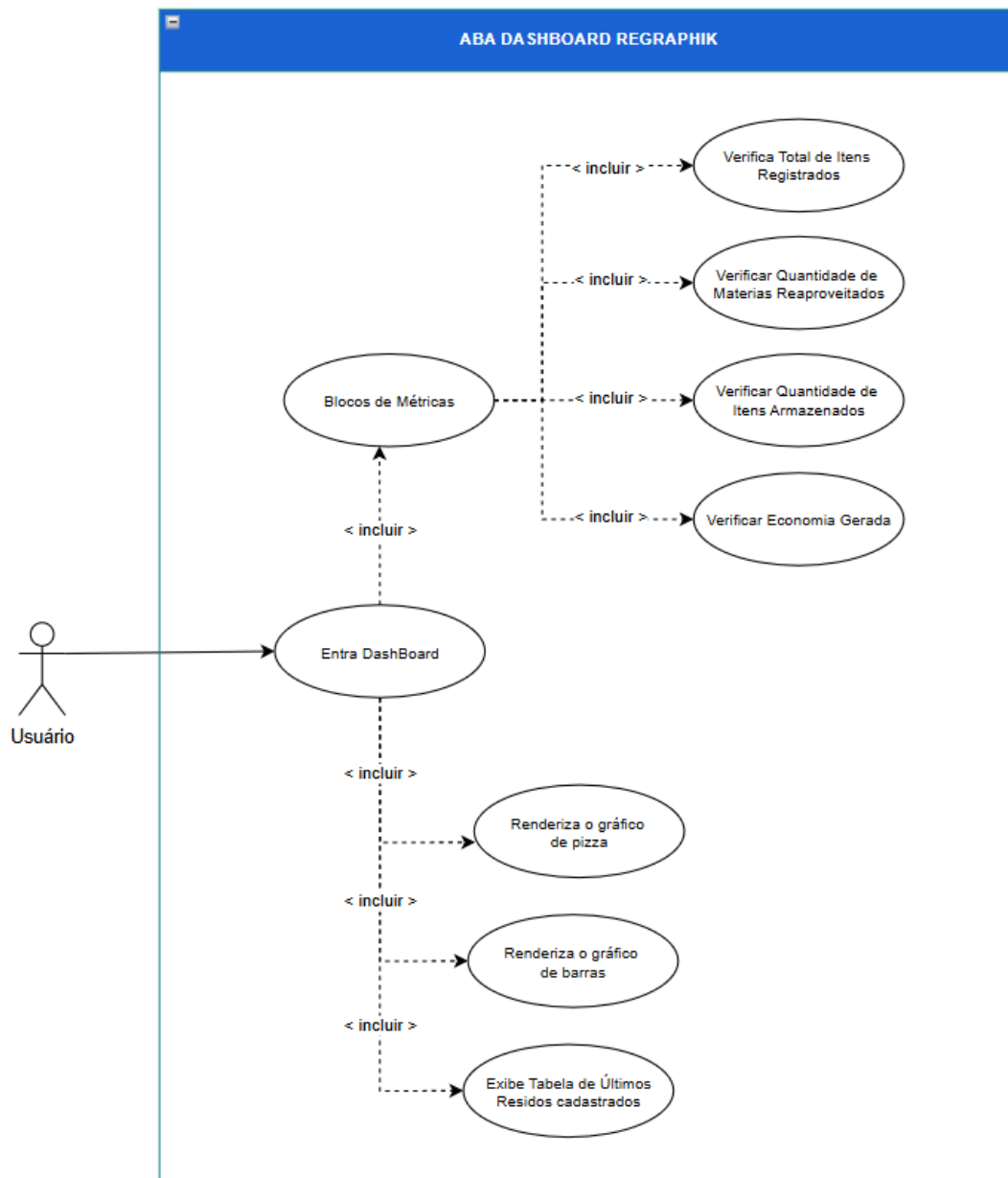


Diagrama de Caso de Uso — Estoque Reverso:

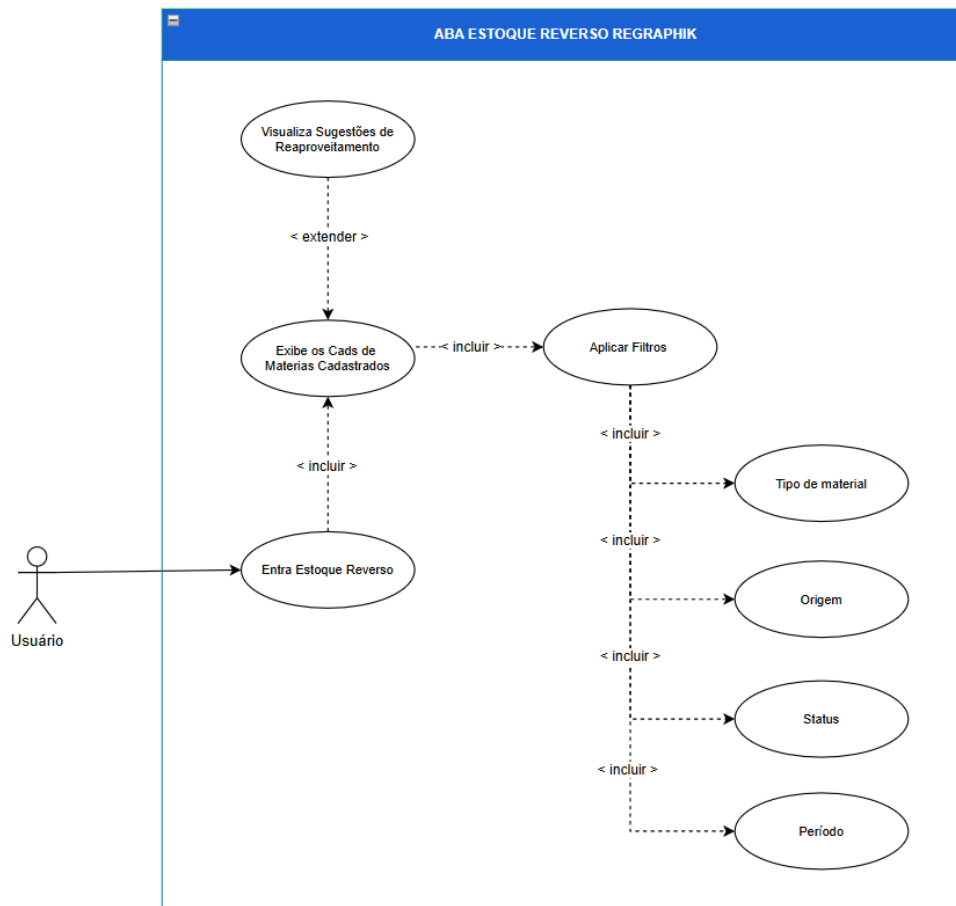


Diagrama de Caso de Uso — Mapa:

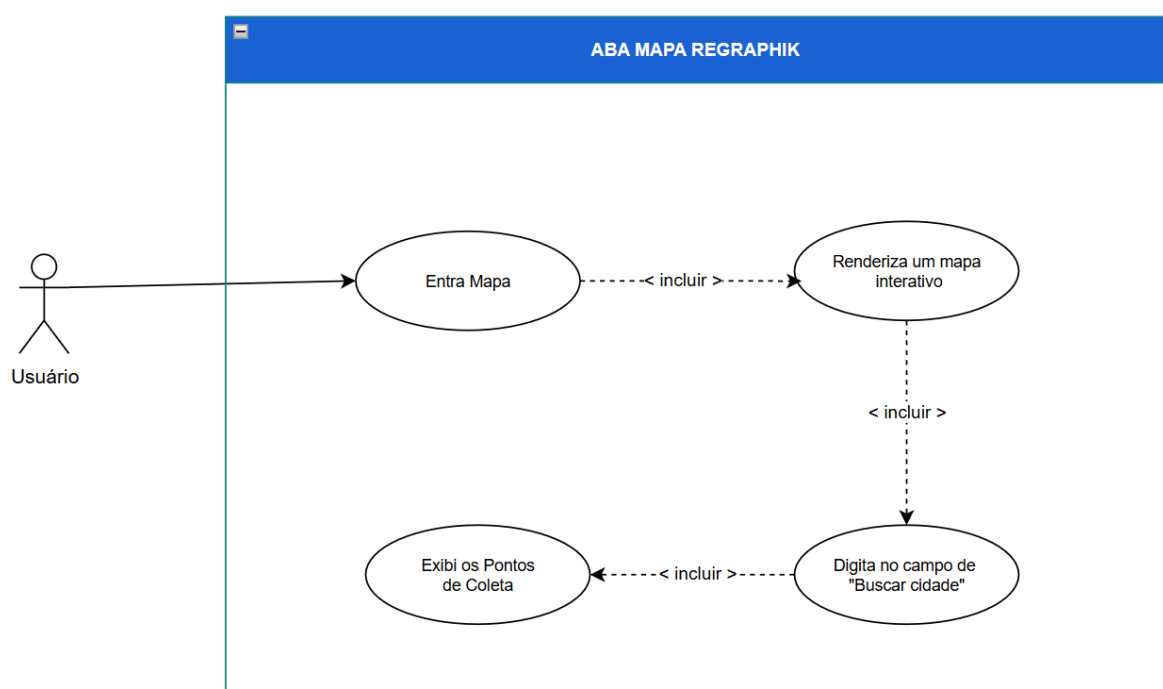


Diagrama de Caso de Uso — Minha Conta:

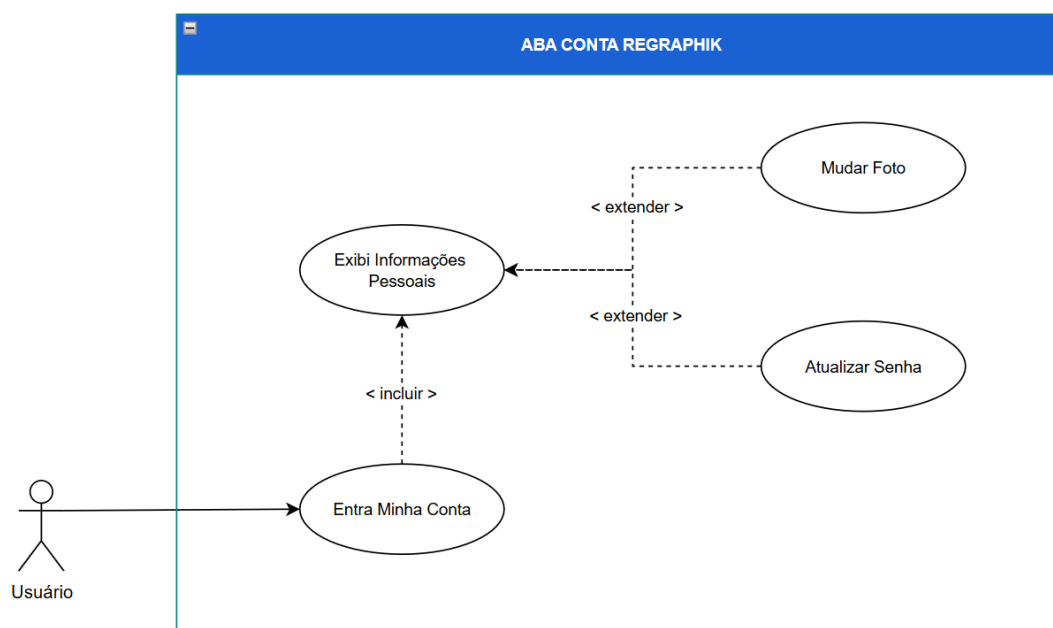
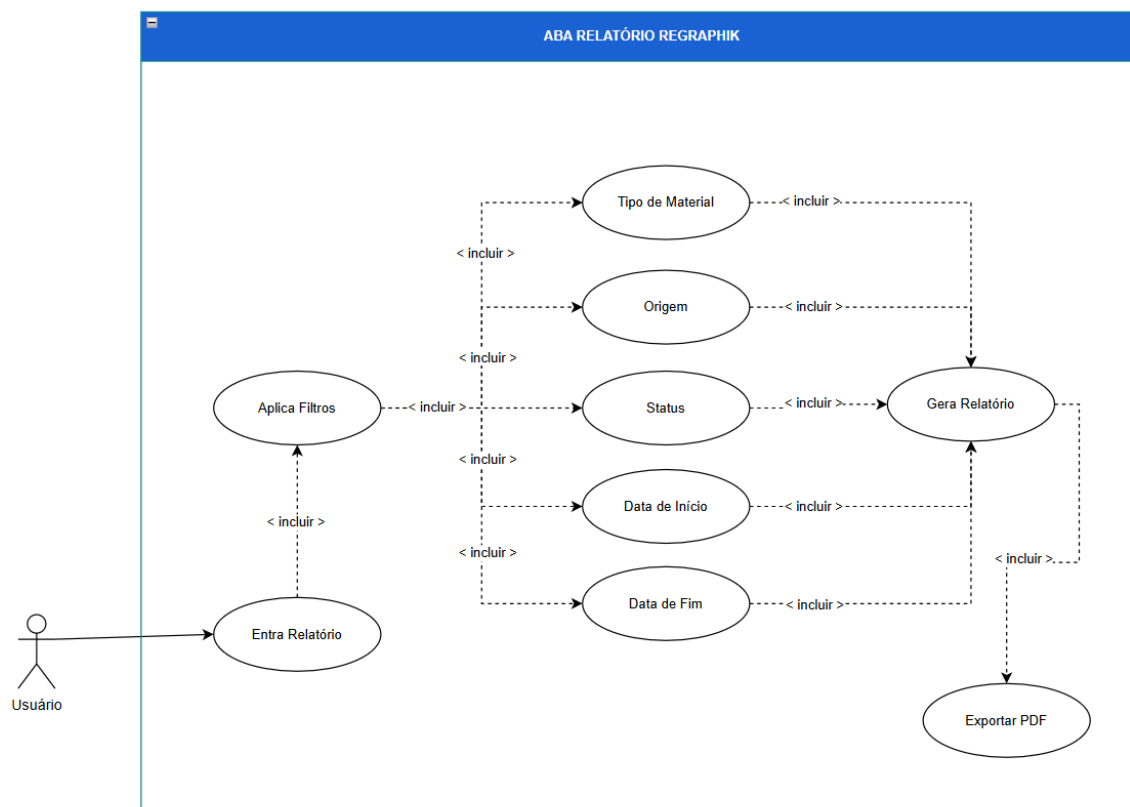


Diagrama de Caso de Uso — Relatórios:



1.3.4.10 Diagrama de Sequência

Diagrama de Sequência — Login:

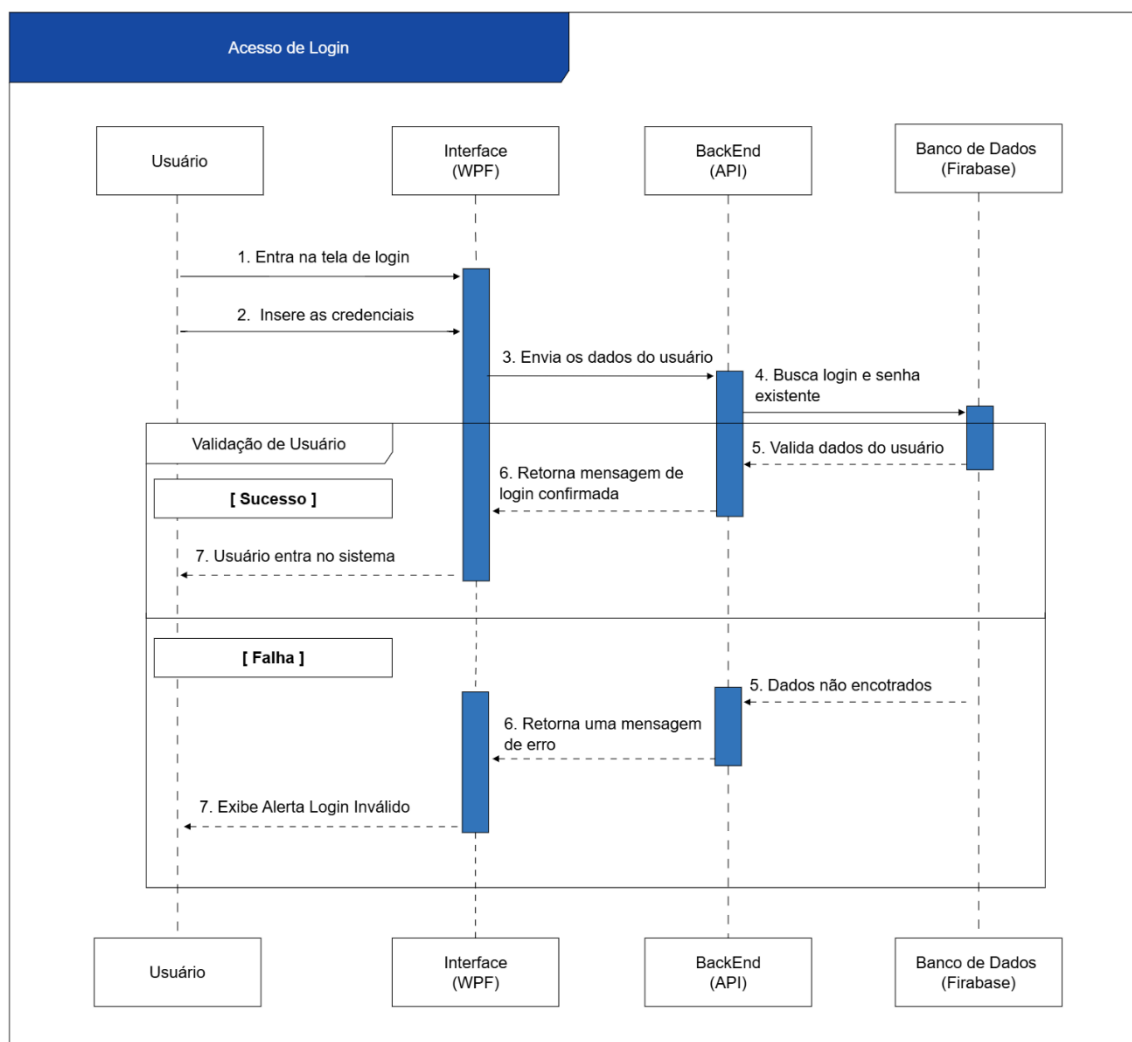


Diagrama de Sequência — Cadastro:

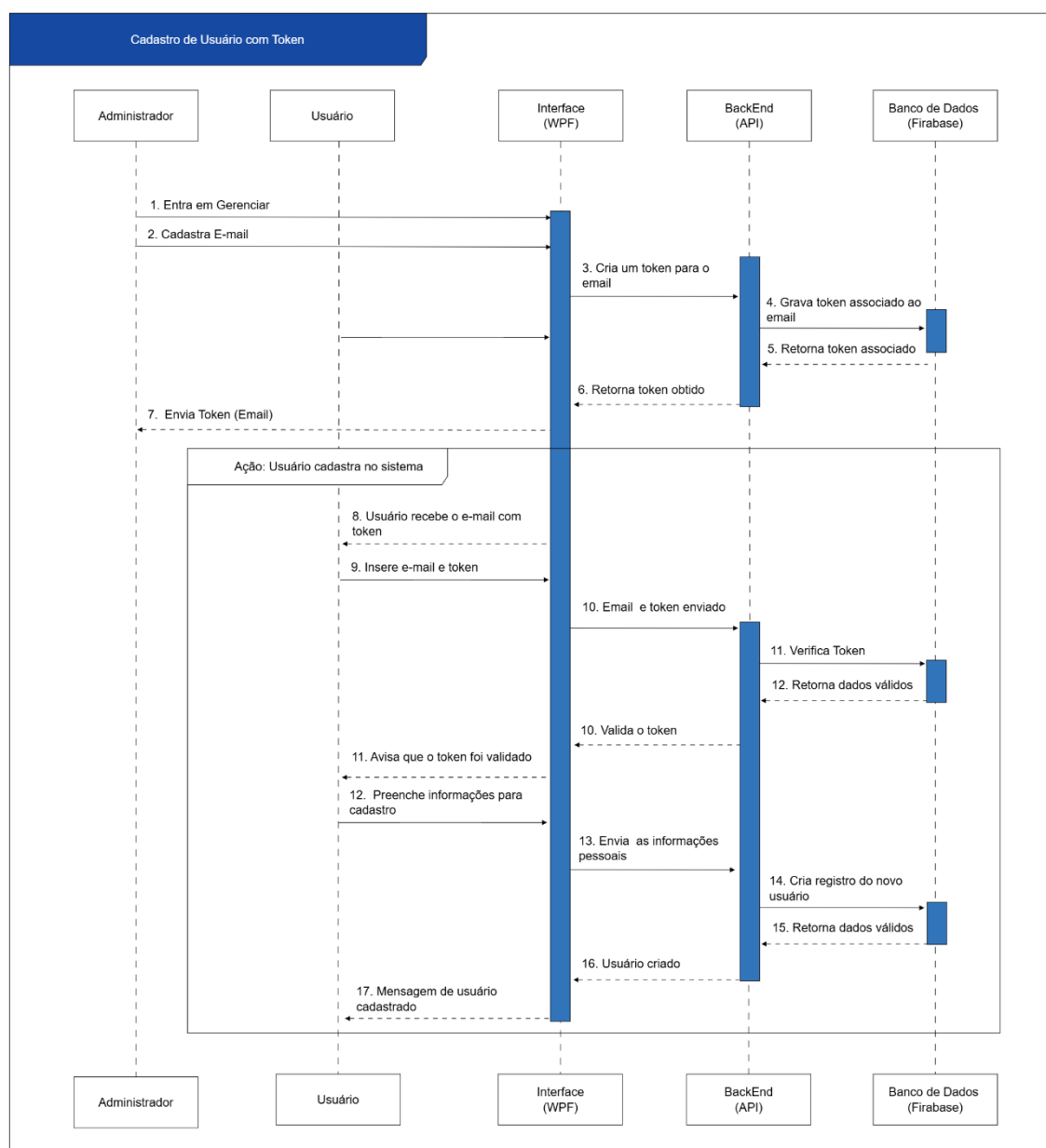


Diagrama de Sequência — Cadastro de Resíduos:

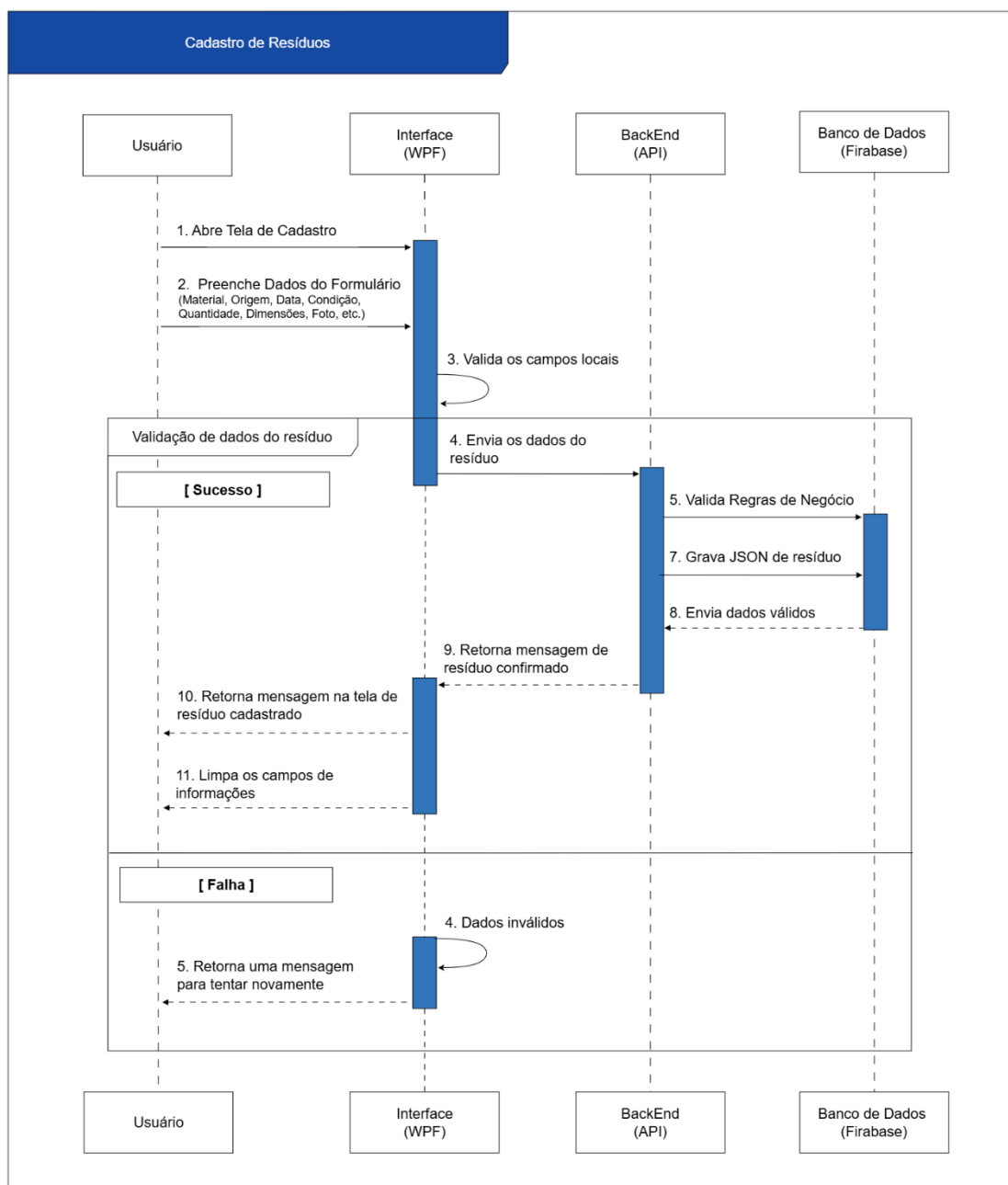


Diagrama de Sequência — Certificação ESG:

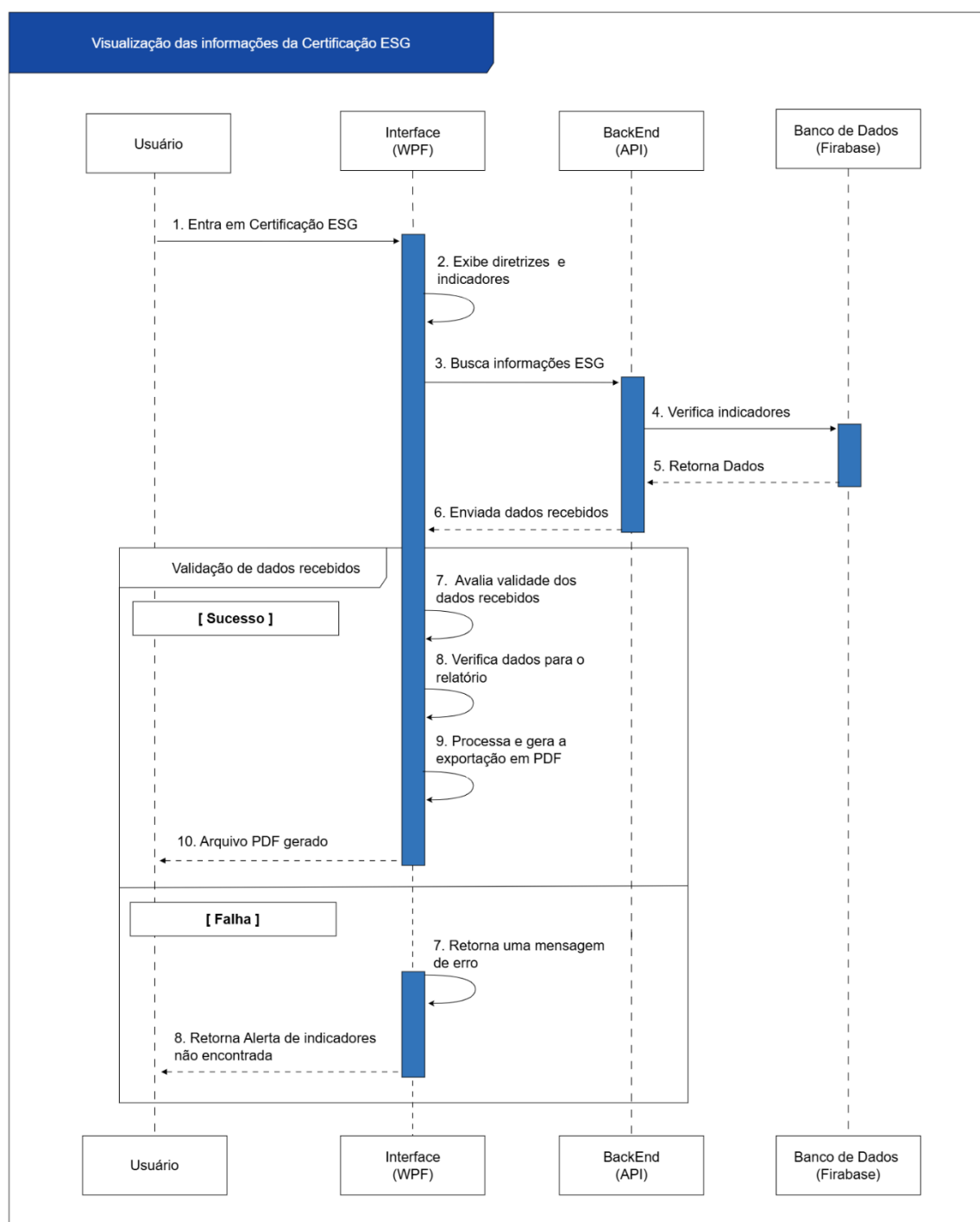


Diagrama de Sequência — Chat:

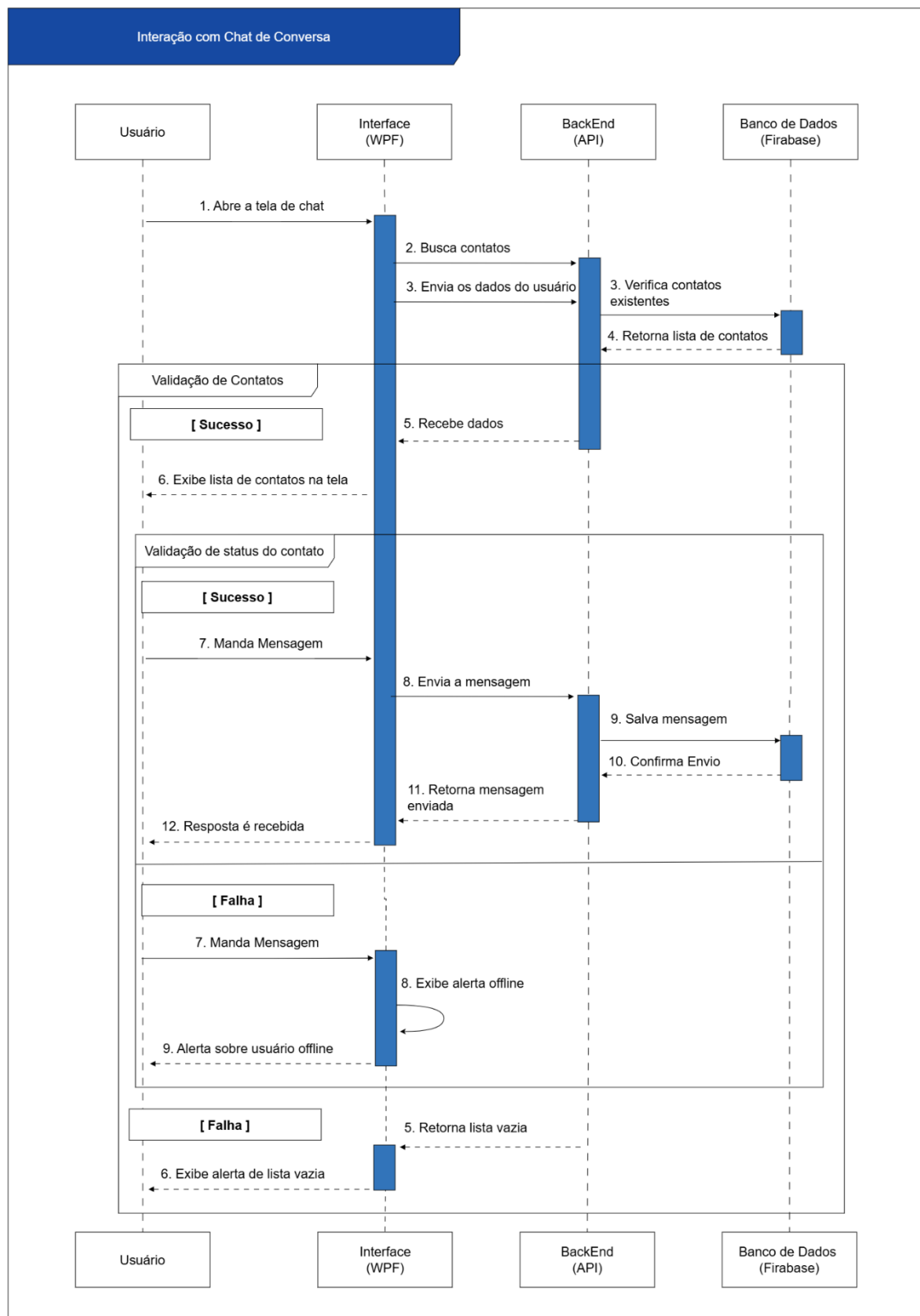
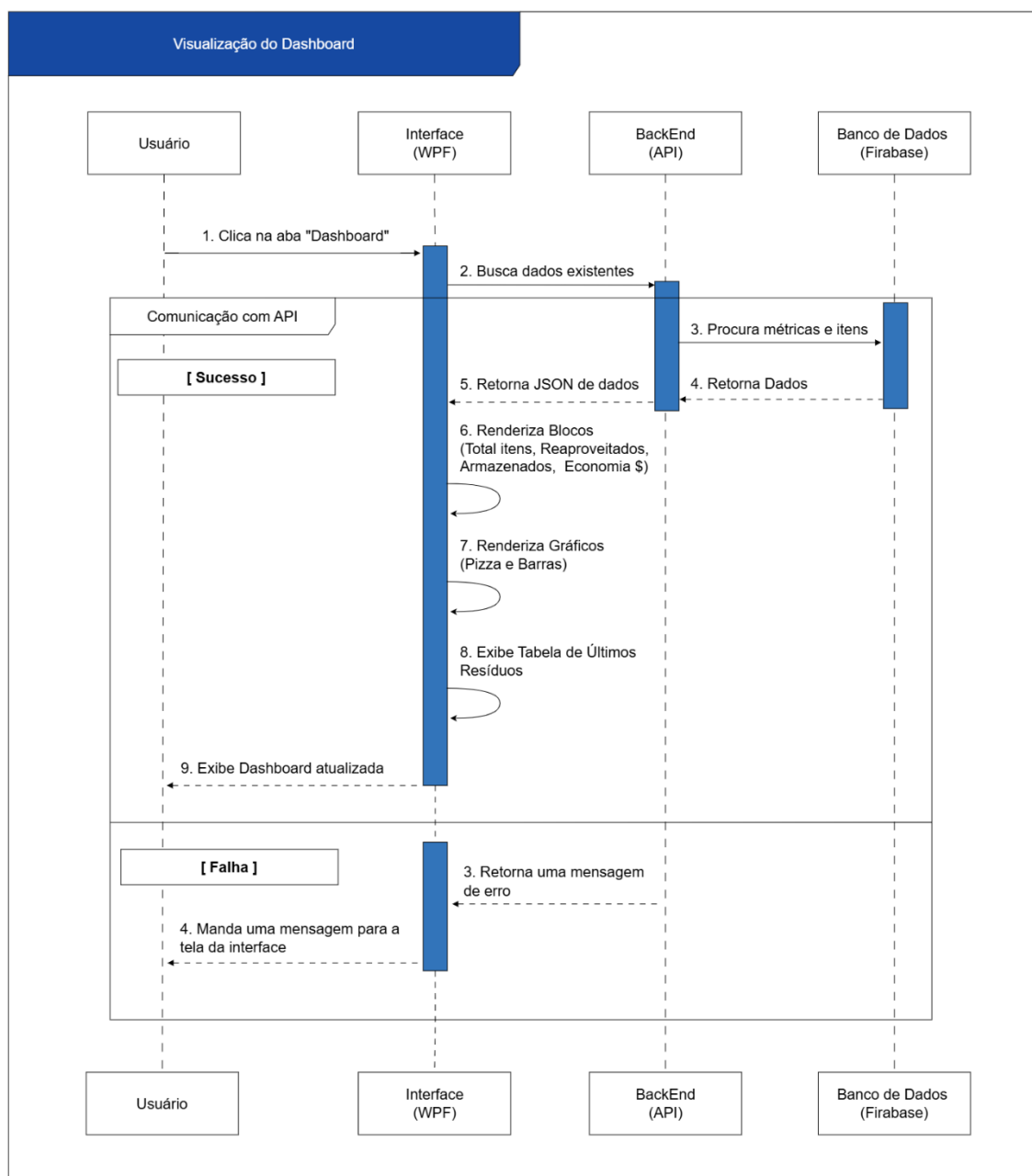


Diagrama de Sequência — Dashborad:

**Diagrama de Sequência — Estoque Reverso:**

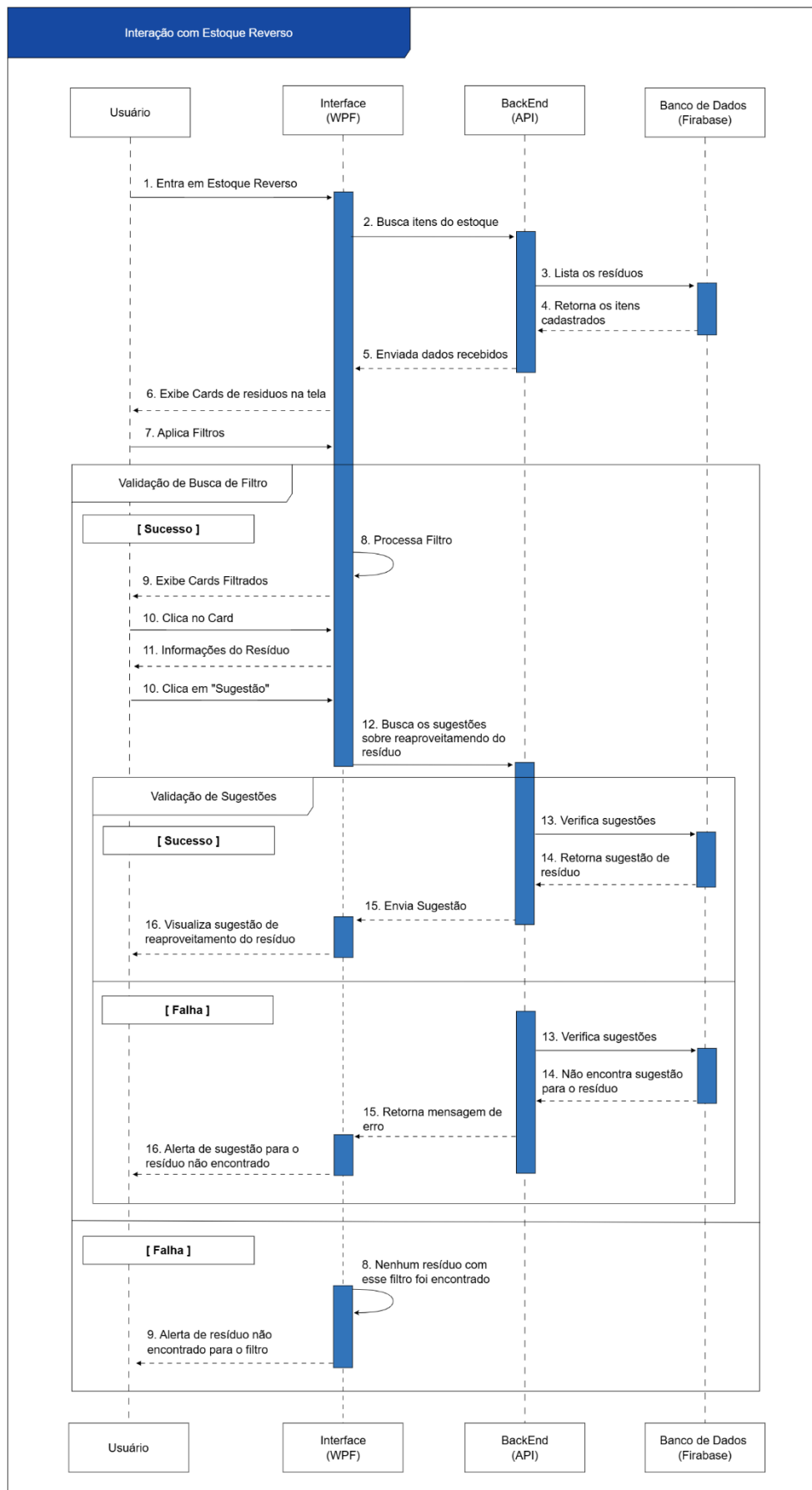


Diagrama de Sequência — Mapa:

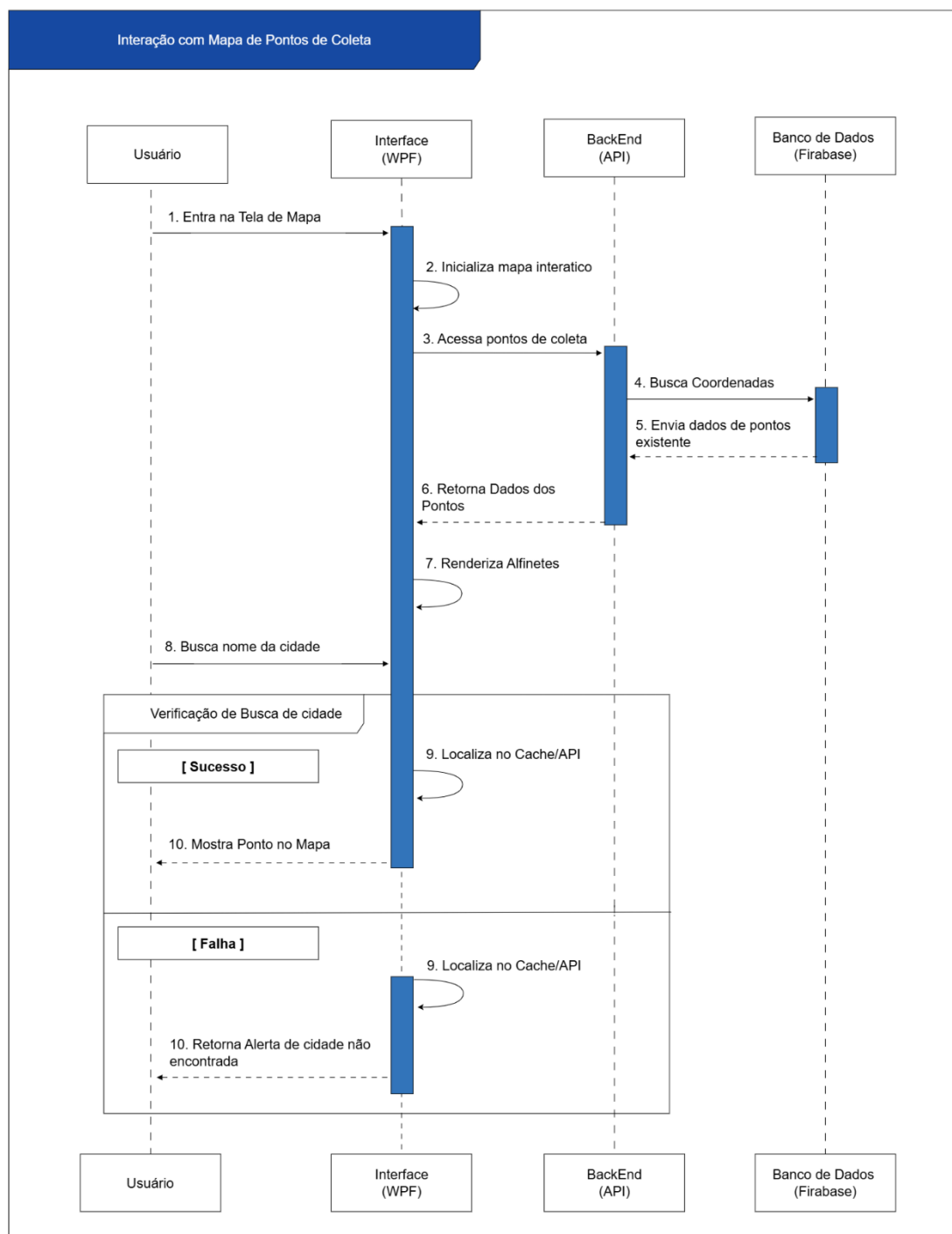
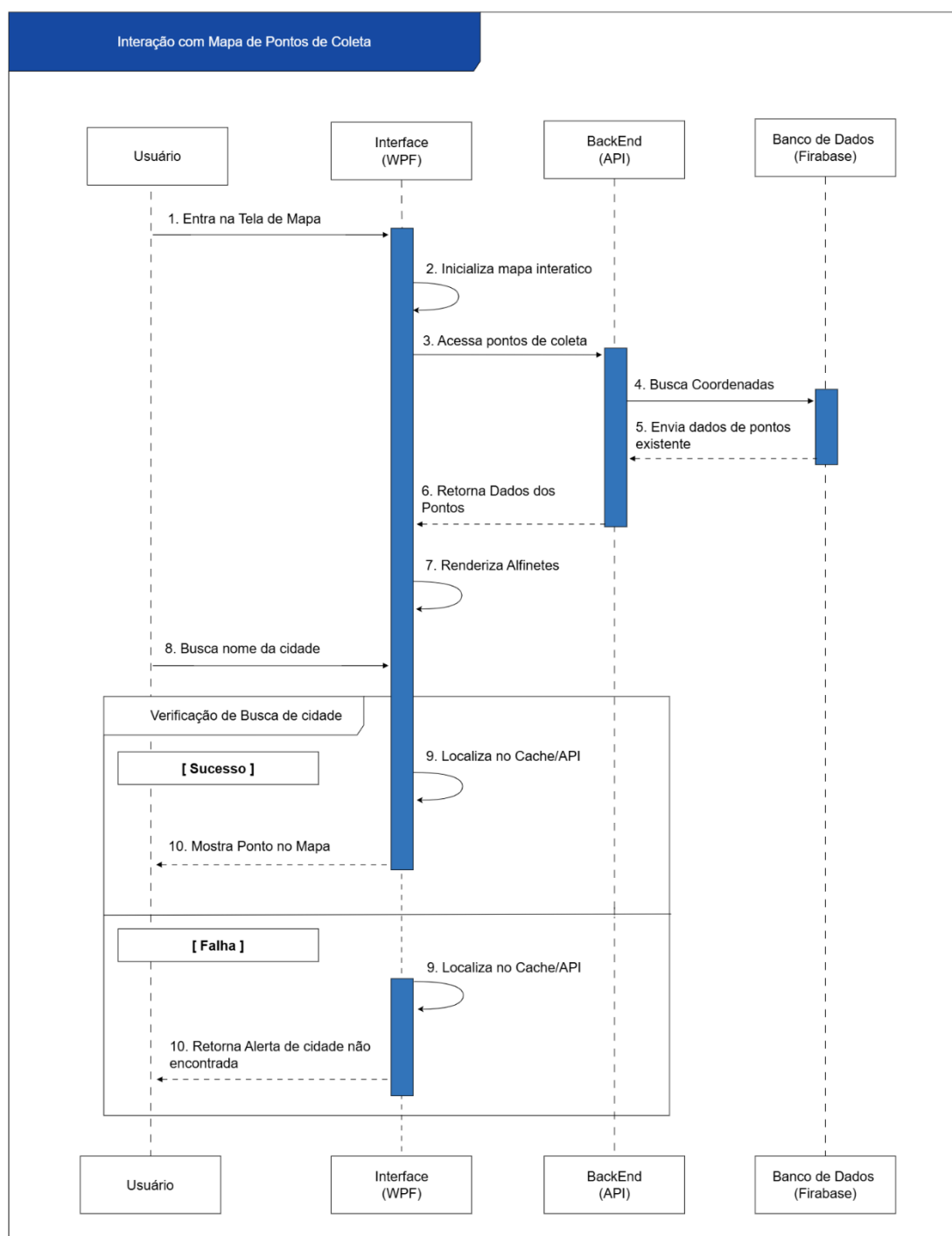


Diagrama de Sequência — Minha Conta:

**Diagrama de Sequência — Perfil de Usuário:**

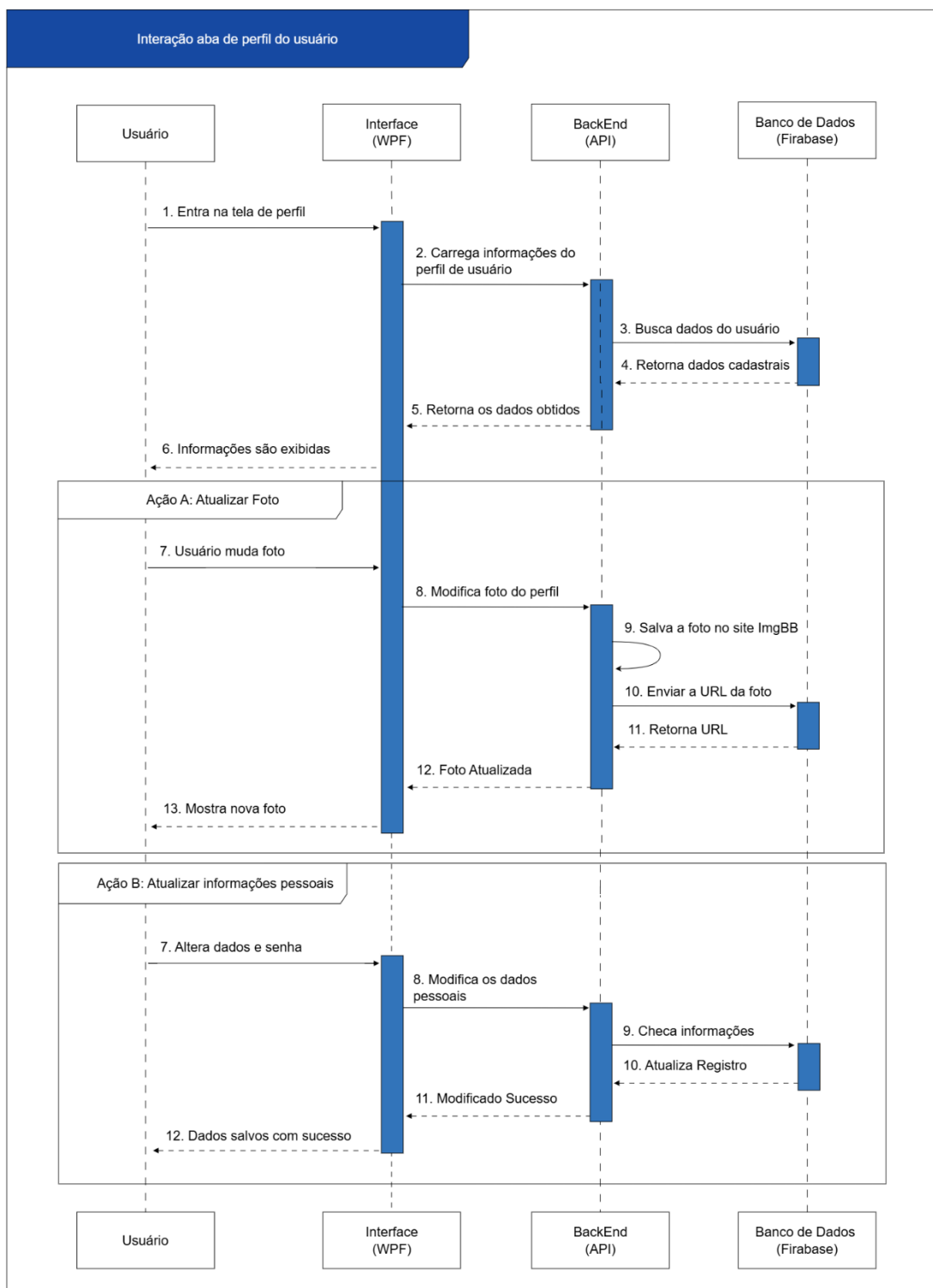
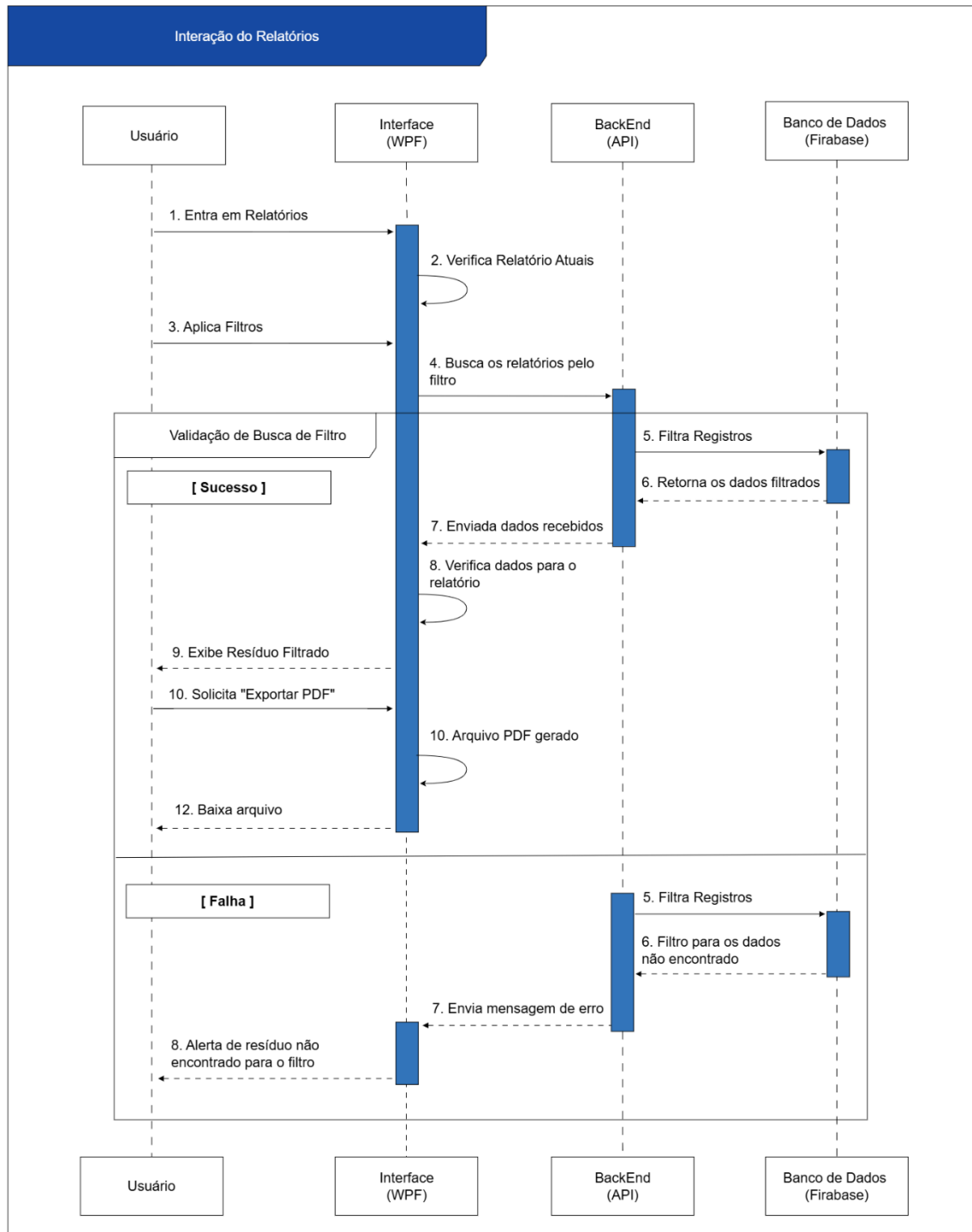


Diagrama de Sequência — Relatórios:



1.3.4.11 Diagrama de Fluxo

Diagrama de Fluxo — Login:

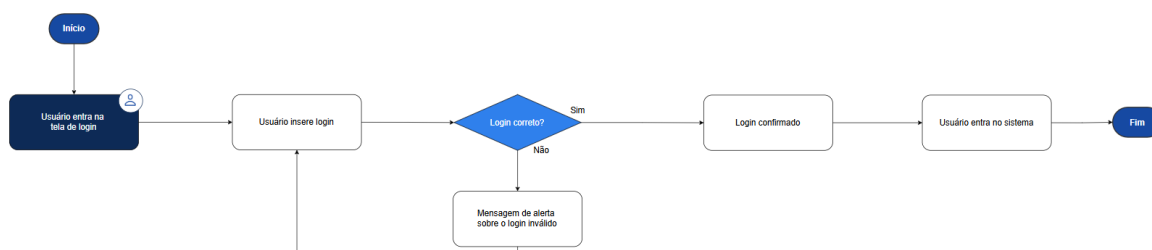


Diagrama de Fluxo — Cadastro:

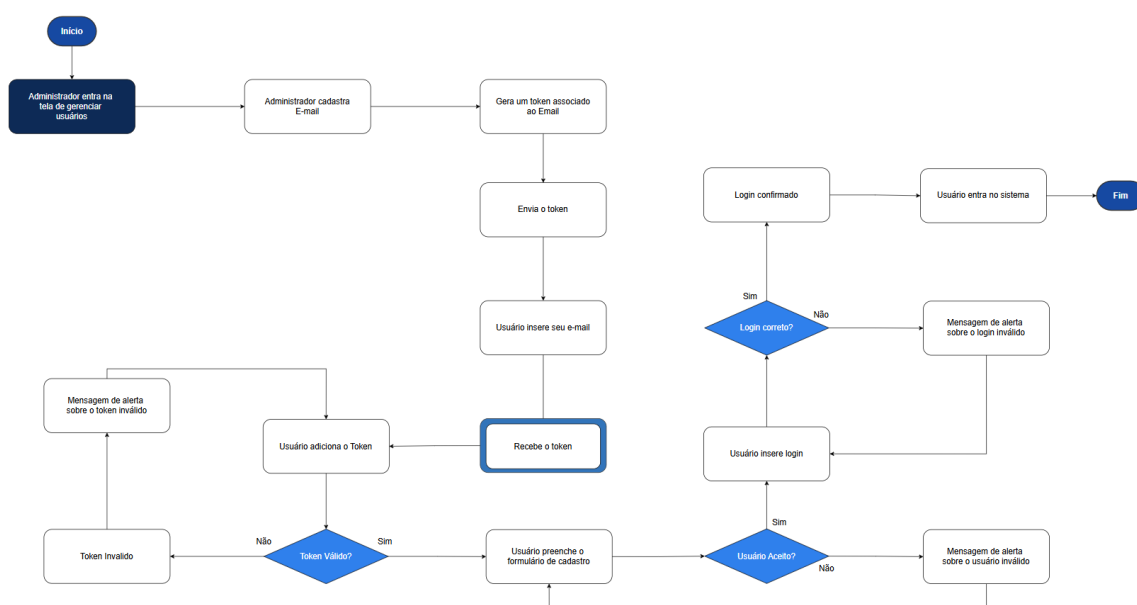


Diagrama de Fluxo — Cadastro de Resíduos:

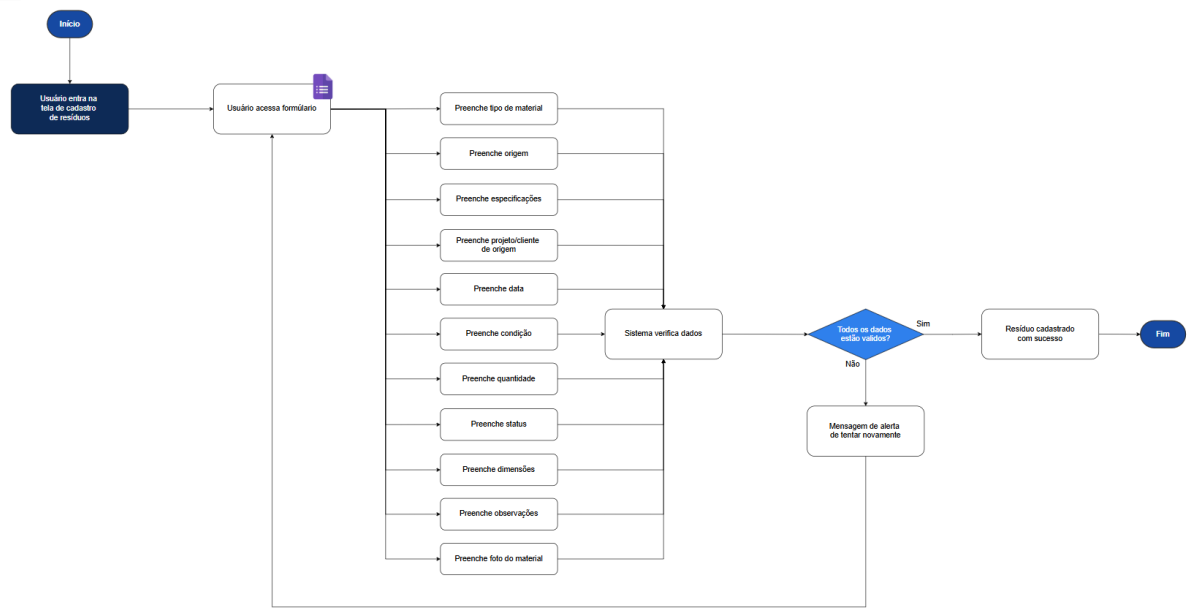


Diagrama de Fluxo — Conta Usuário:

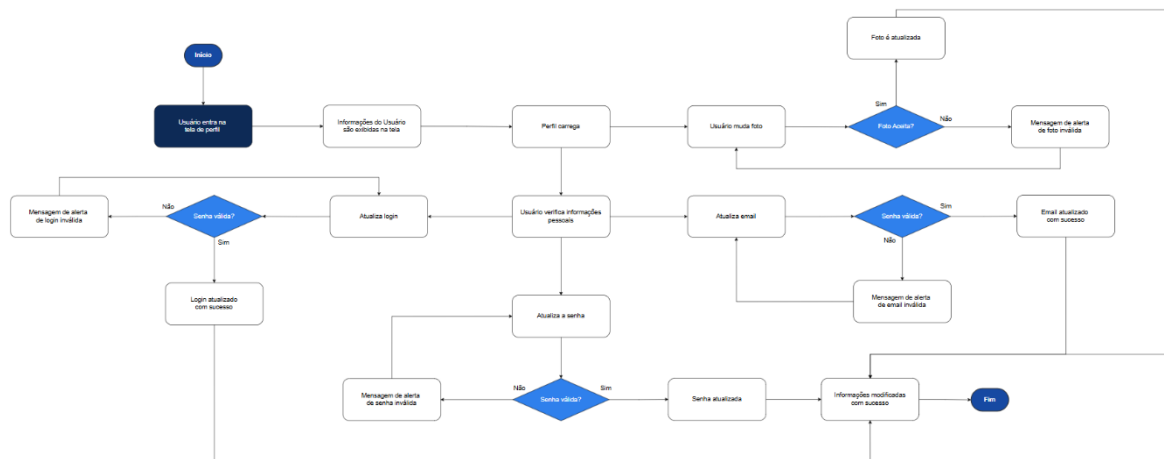


Diagrama de Fluxo — Dashboard:

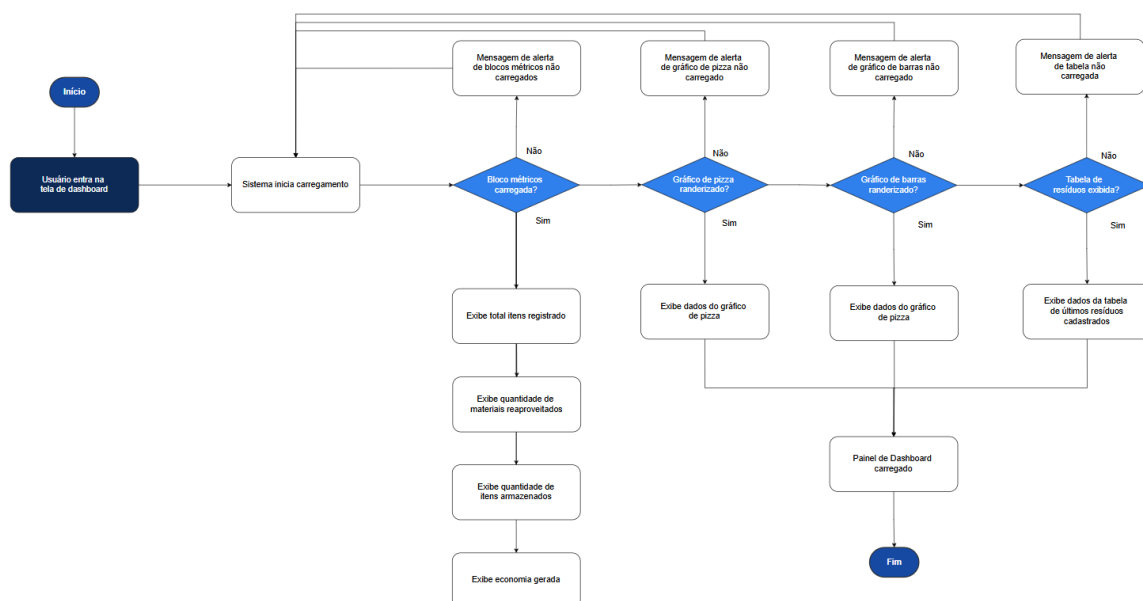


Diagrama de Fluxo — ESG:

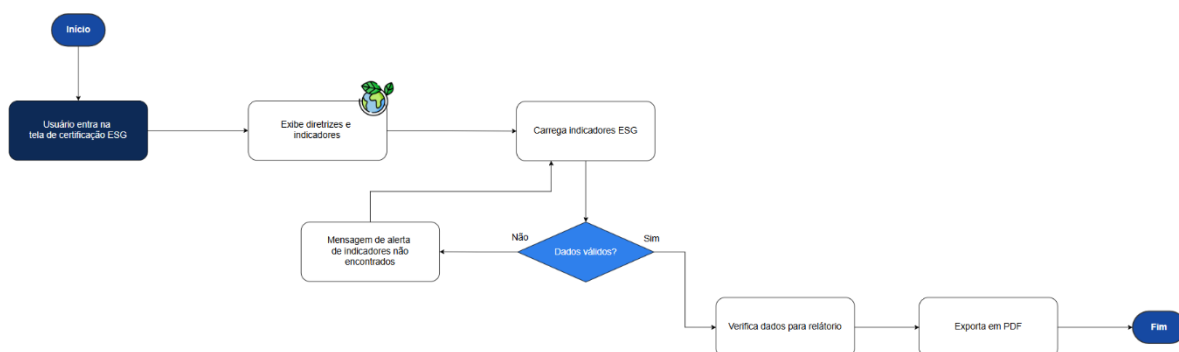


Diagrama de Fluxo — Estoque Reverso:

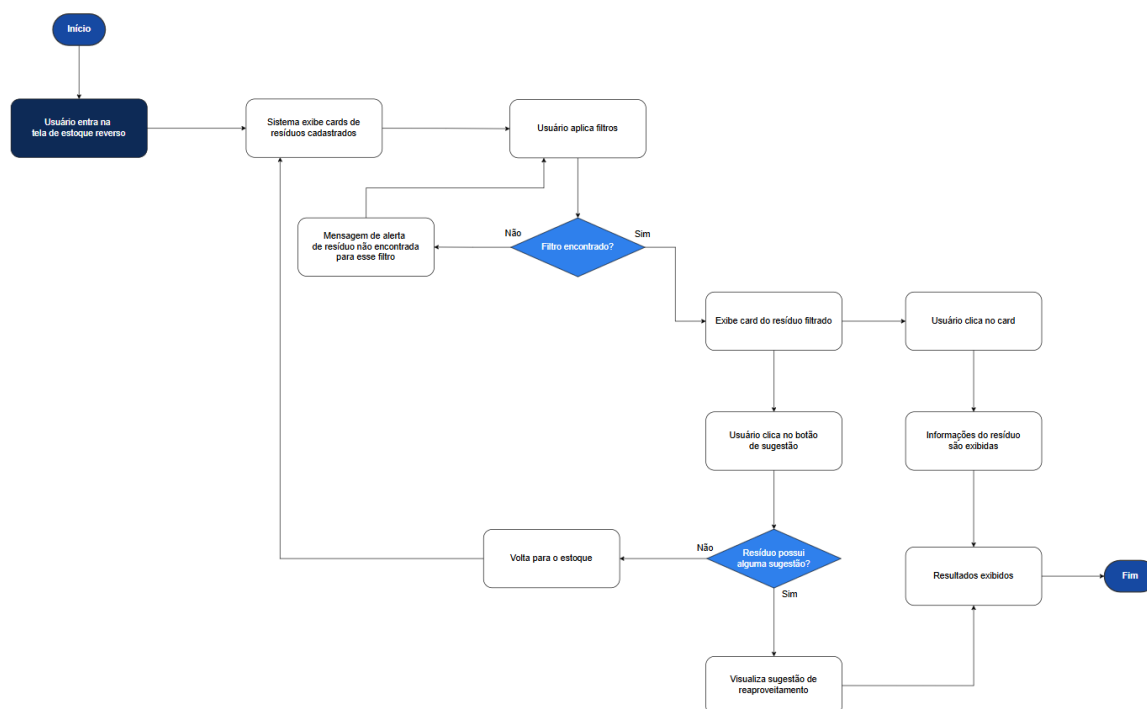


Diagrama de Fluxo — Mapa:

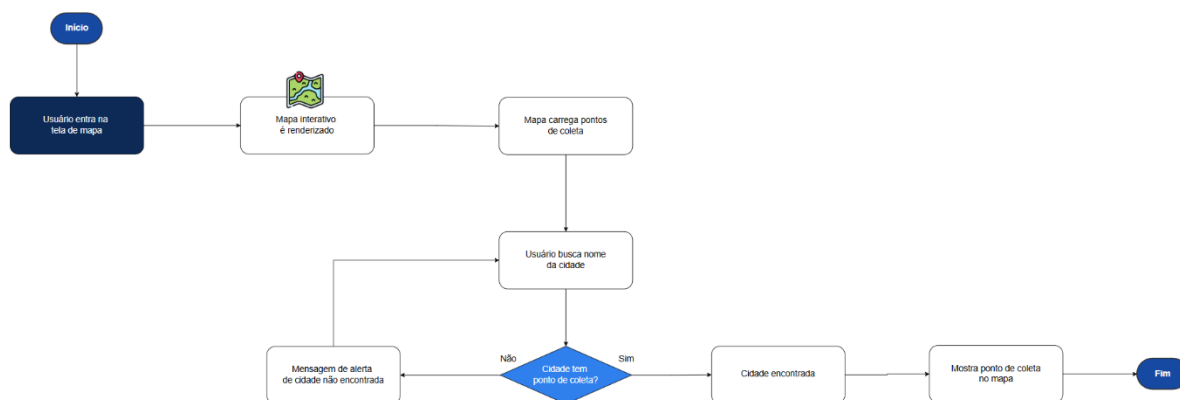
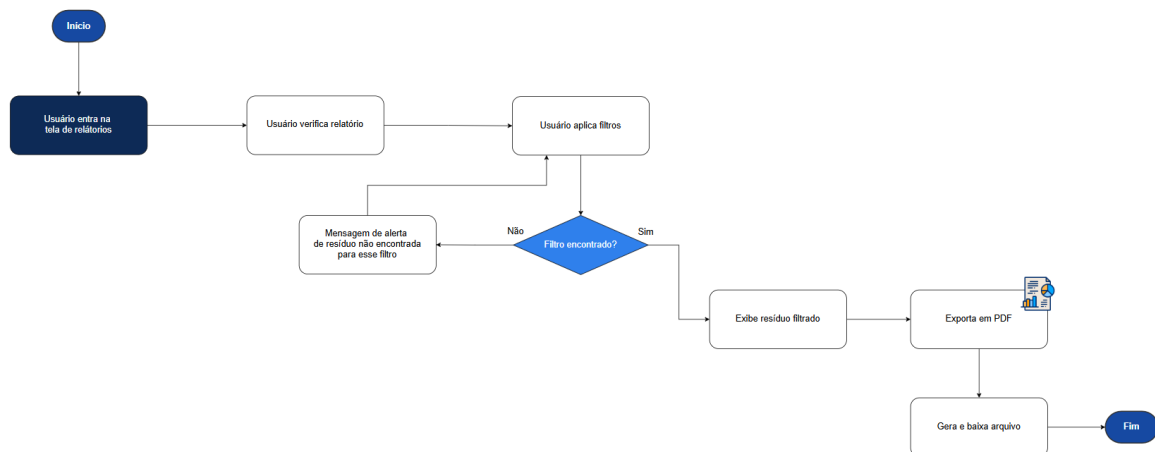


Diagrama de Fluxo — Relatórios:

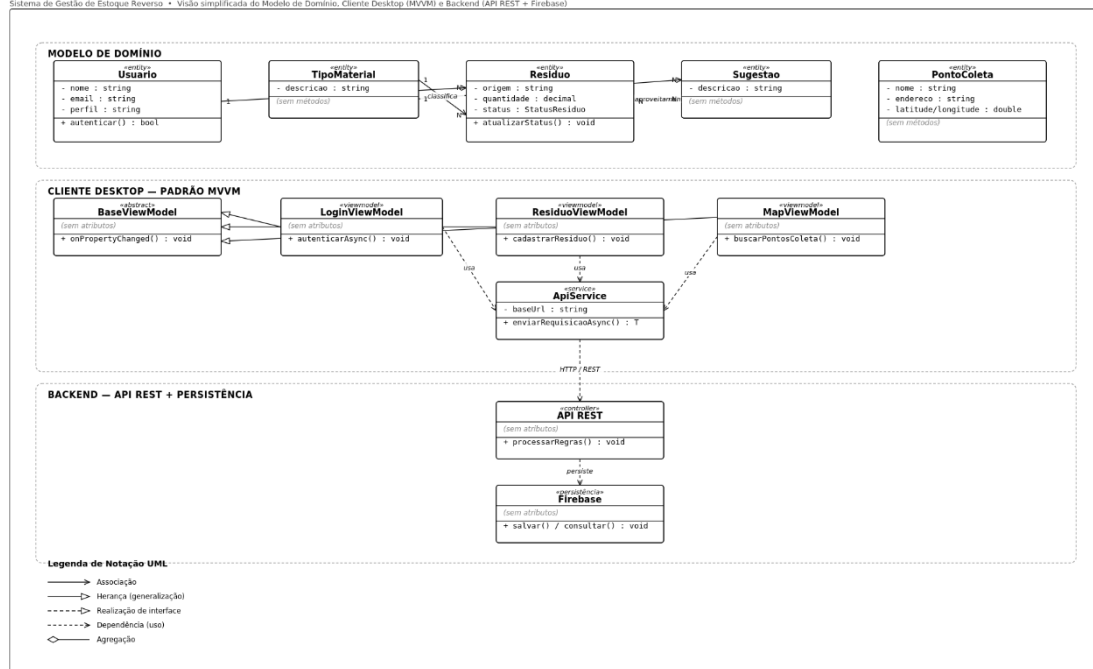


1.3.4.12 Diagrama de Classe

Como ler o diagrama: as classes estão organizadas em três camadas — Modelo de Domínio (entidades persistidas no Firebase), Cliente Desktop MVVM (ViewModels, Commands e Services consumidos pela WPF View) e API REST / Backend (Controllers ASP.NET Core e o contexto de persistência). As setas seguem a notação UML padrão: linha cheia com seta aberta indica associação/navegação; triângulo vazado indica herança ou realização de interface (esta última tracejada); seta tracejada indica dependência de uso; e losango vazado indica agregação.

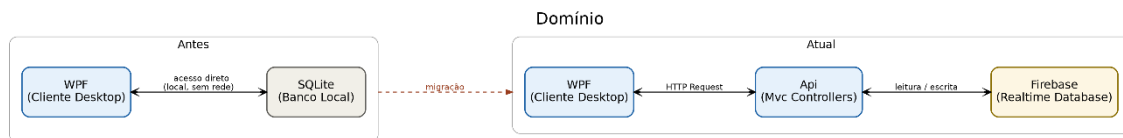
Diagrama de Classes — ReGraphik (Visão Macro)

Sistema de Gestão de Estoque Reverso • Visão simplificada do Modelo de Domínio, Cliente Desktop (MVVM) e Backend (API REST + Firebase)



1.3.4.14 Modelagem de Domínio

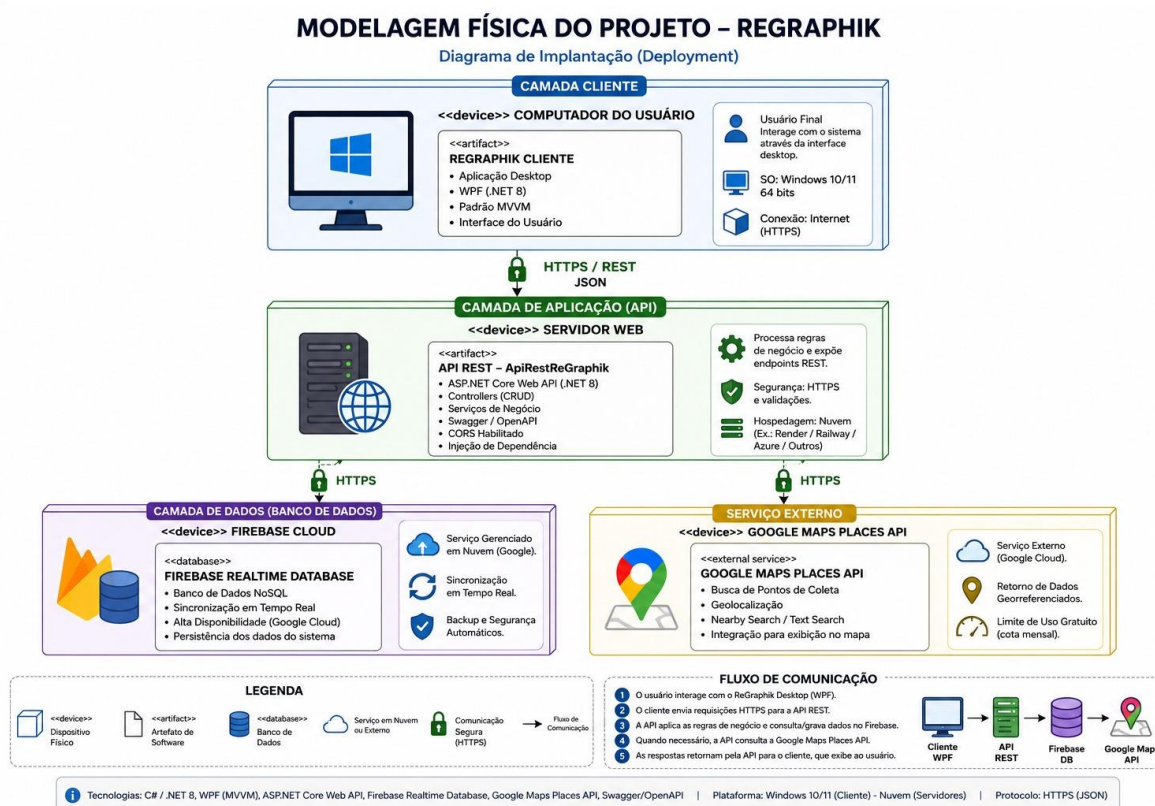
Evolução da arquitetura de comunicação do ReGraphik: originalmente, o cliente WPF acessava um banco SQLite local diretamente. Atualmente, o WPF se comunica via HTTP com a API (MVC Controllers), que é responsável por toda leitura e escrita no Firebase Realtime Database.



Por que migramos do SQLite para API + Firebase:

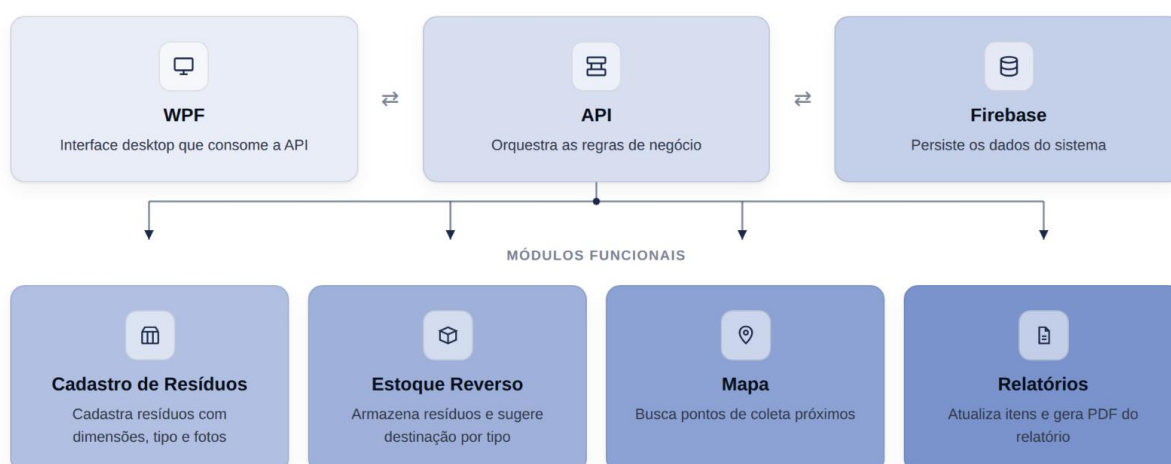
- **Dados isolados por máquina:** o SQLite era um banco local — cada instalação do WPF tinha seu próprio arquivo de banco, sem sincronização entre usuários da mesma empresa. Um resíduo cadastrado por um usuário não aparecia para os demais.
- **Login incompatível:** o sistema de login original, baseado em SQLite, não era compatível com o modelo de autenticação adotado no Firebase, o que exigiu recriar as telas de login/cadastro do zero.
- **Necessidade de dados centralizados e compartilhados:** o negócio exige que todos os usuários de uma mesma empresa vejam o mesmo estoque reverso em tempo real — algo que um banco local por máquina não permite.
- **API como camada intermediária:** em vez do WPF acessar o Firebase diretamente, a API concentra as regras de negócio, validações e autenticação, evitando que a lógica fique espalhada ou duplicada entre múltiplos clientes desktop.

1.3.4.15 Modelagem Física do sistema



1.3.4.16 Modelagem Lógica

O sistema é composto por um cliente desktop em WPF e uma API, que se comunicam entre si em via de mão dupla. A API é responsável por persistir e consultar os dados no Firebase, retornando as informações para o WPF exibir ao usuário.



- **Cadastro de Resíduos** — permite cadastrar os resíduos informando dimensões, tipo e fotos do item.

- **Estoque Reverso** — armazena os resíduos que foram cadastrados e sugere possíveis destinações de acordo com o tipo de cada resíduo.
- **Mapa** — busca e exibe os pontos de coleta disponíveis.
- **Relatórios** — atualiza os itens cadastrados e permite baixar um PDF com o relatório consolidado desses resíduos.

1.3.4.17 Histórico de Evolução e Atualização do Projeto

Fase de Inicialização e Infraestrutura Base (Setup Inicial)

Estruturação da solução multicamadas no Visual Studio, separando o backend de microserviços (ApiRestReGraphik) do cliente rico desktop (ReGraphik). Implementação da infraestrutura MVVM nativa, com a classe abstrata BaseViewModel (INotifyPropertyChanged) e a classe genérica RelayCommand, substituindo manipuladores de eventos em code-behind por binds declarativos em XAML.

Fase de Amadurecimento do Frontend e Integração Geográfica

Codificação das interfaces com bindings bidirecionais para as telas de autenticação, painel de resíduos e módulo cartográfico. Acoplamento do Leaflet.js com WebView2, viabilizando mapas interativos no ambiente Windows Desktop. Desenvolvimento do GooglePlacesService, estruturado sob a interface IPlacesService, isolando as chaves de API externas.

Fase de Construção do Backend, Persistência e Sincronização em Nuvem

Migração para persistência global em nuvem via Firebase SDK, com a classe de contexto DbReGraphik.cs conectando a API às chaves do Google Service Account (OAuth 2.0). Desenvolvimento dos 5 controllers REST (Usuario, PontosColeta, Residuos, Sugestao, SugestaoResiduos). Implementação de mecanismo anti-redundância no endpoint /api/PontosColeta/google, otimizando custos operacionais. Configuração do Swagger UI (Swashbuckle), expondo documentação interativa completa em /swagger.

1.3.4.18 Regras de Negócio Detalhadas

Autenticação e Cadastro

- Pré-cadastro gera token numérico de 6 dígitos e cria registro com Ativo = false; o token é enviado por e-mail.
- Validação do token marca o usuário como Ativo = true no pré-cadastro (ApiConfig).
- Finalização do cadastro exige token autorizado em memória (ApiRestReGraphik); ao criar o usuário, define-se Id (GUID), DataCadastro (UTC) e Perfil (padrão 'User').
- Login é realizado via POST /api/Usuario/login (login + senha).

Validação de Dados

- CPF validado localmente por algoritmo oficial de dígitos verificadores e formatado como 000.000.000-00.
- Upload de arquivos restrito a extensões de imagem (jpg, jpeg, png, bmp).

Resíduos, Sugestões e Chat

- CRUD de resíduos com campos obrigatórios validados antes do cadastro; estoque reverso visualizado como cards com cores por status.
- Relatórios calculam TotalResiduos, PesoTotal, Reaproveitados e ValorEconomico, com exportação via PrintDialog/PDF.
- Sugestões são filtradas por tipo de material (com correspondência parcial e bidirecional) e podem ser aplicadas, gerando registro em SugestoesResiduos com data de aplicação.
- Chat interno persistido no Firebase, com identificador de conversa determinístico (ordenação alfabética dos IDs) e marcação de mensagens como lidas.

Pontos de Coleta e Configuração Local

- O endpoint /api/PontosColeta/google evita duplicidade de cidades e faz cache dos pontos antes de requisições externas pagas.
- A foto de perfil é salva localmente (%AppData%/ReGraphik/config.txt) e recarregada entre sessões.

1.3.5 VIABILIDADE TÉCNICA

A análise de viabilidade técnica avalia se o sistema ReGraphik pode ser desenvolvido, implantado e mantido com os recursos tecnológicos disponíveis, considerando o contexto acadêmico do projeto e a realidade das empresas do setor gráfico a que se destina.

1.3.5.1 Infraestrutura e Tecnologias

Componente	Licença	Maturidade	Observação
C# / .NET 8+	MIT (open-source)	Alta	Plataforma Microsoft com suporte LTS garantido
ASP.NET Core	MIT (open-source)	Alta	Framework web maduro, usado em produção globalmente
WPF	MIT (open-source)	Alta	Framework nativo Windows, base de código estável
Firebase Realtime DB	Gratuito (plano Spark)	Alta	Plataforma Google com 99,95% de SLA
Google Maps Places API	Gratuito até US\$200/mês	Alta	API estável, versão nova em produção
Leaflet.js	BSD-2-Clause	Alta	Biblioteca de mapas mais utilizada do mundo
Swagger/OpenAPI	Apache 2.0	Alta	Padrão de mercado para documentação de APIs REST
Git / GitHub	Gratuito (plano Free)	Alta	Plataforma de controle de versão líder de mercado

Requisitos mínimos de hardware: processador Intel Core i3 (2 GHz ou superior); 4 GB de RAM (8 GB recomendado para desenvolvimento); 500 MB de armazenamento local (dados persistidos em nuvem); Windows 10/11 de 64 bits (obrigatório para o cliente WPF); e conexão com a internet para Firebase e Google Maps.

Requisitos de software: .NET 8 Runtime ou superior; Visual C++ Redistributable; e o componente WebBrowser/WebView2, necessário para a renderização do mapa via Leaflet.js.

1.3.5.2 Riscos Técnicos e Mitigações

Risco	Prob.	Impacto	Mitigação
Cota gratuita do Firebase excedida	Média	Alto	Migrar para plano Blaze (pay-as-you-go) ou banco relacional próprio
Cota gratuita da Google Maps API excedida	Média	Médio	Cache de resultados no Firebase e limite de chamadas por sessão
Dependência exclusiva de Windows (WPF)	Baixa	Médio	Desenvolver versão web (Blazor) ou mobile (.NET MAUI)
Ausência de autenticação JWT na API	Alta	Alto	Implementar JWT como prioridade na próxima sprint
Perda de conectividade com a internet	Baixa	Alto	Cache local (SQLite) para funcionamento offline parcial
Quebra de contrato da Google Maps API	Muito Baixa	Médio	Interface IPlacesService abstrai o provedor de mapas

1.3.5.3 Escalabilidade

- A API REST em ASP.NET Core pode ser hospedada em múltiplas instâncias (horizontal scaling) com load balancer, sem alteração de código;
- O Firebase Realtime Database escala automaticamente conforme a demanda;
- O padrão MVVM no cliente WPF permite adicionar novos módulos sem modificar os existentes;
- A interface IPlacesService abstrai o provedor de mapas, permitindo substituição sem impacto nas ViewModels;
- A separação entre Controllers e Services facilita a extração de microserviços futuros.

1.3.5.4 Stack Tecnológica — Resumo

Camada	Tecnologia	Versão / Detalhe
Linguagem	C# (.NET)	.NET 8 / .NET 9
Frontend Desktop	WPF	Windows-only
Padrão de Projeto	MVVM	Padrão GoF
Backend / API	ASP.NET Core Web API	REST / JSON
Banco de Dados	Firebase Realtime Database	NoSQL — Google
Autenticação Firebase	Google Service Account (JSON)	OAuth 2.0
Mapas / Geolocalização	Google Maps Places API + Leaflet.js	WebBrowser/WebView2
Documentação da API	Swagger / OpenAPI	Swashbuckle
Controle de Versão	Git / GitHub	Histórico consistente de commits
IDE	Visual Studio 2022	Community Edition

1.3.6 VIABILIDADE ECONÔMICA

A análise de viabilidade econômica examina os custos envolvidos no desenvolvimento e operação do ReGraphik, os benefícios financeiros esperados com sua adoção e o retorno sobre o investimento para as empresas do setor gráfico que o utilizarem.

1.3.6.1 Custos de Desenvolvimento

Por tratar-se de um projeto acadêmico, os custos de desenvolvimento foram essencialmente de tempo e capacitação da equipe. A tabela projeta esses custos em valores de mercado, considerando a remuneração média de desenvolvedores júnior/pleno no Brasil:

Item	Horas Est.	Valor/Hora	Custo Total
Levantamento de requisitos e Mini Mundo	20 h	R\$ 60,00	R\$ 1.200,00
Modelagem de dados (Conceitual, Lógico, Físico)	30 h	R\$ 60,00	R\$ 1.800,00
Desenvolvimento da API REST	80 h	R\$ 80,00	R\$ 6.400,00
Desenvolvimento do cliente WPF (MVVM)	100 h	R\$ 80,00	R\$ 8.000,00
Integração Firebase + Google Maps API	40 h	R\$ 80,00	R\$ 3.200,00
Testes, correções e refatoração	30 h	R\$ 60,00	R\$ 1.800,00
Documentação técnica e TCC	30 h	R\$ 50,00	R\$ 1.500,00
TOTAL	330 h	—	R\$ 23.900,00

Por ser um projeto de TCC com equipe de cinco estudantes, o custo real de desenvolvimento foi zero em termos monetários diretos, representando apenas o investimento de tempo e aprendizado da equipe.

1.3.6.2 Custos Operacionais

Serviço	Custo Mensal Estimado	Observação
Firebase (plano Spark)	R\$ 0,00	Gratuito até 1 GB de armazenamento e 10 GB/mês de transferência
Firebase (plano Blaze)	R\$ 25,00 – R\$ 150,00	Cobrado por uso acima do plano gratuito; pay-as-you-go
Google Maps Places API	R\$ 0,00 – R\$ 100,00	Crédito de US\$ 200/mês cobre ~13.000 buscas/mês

Serviço	Custo Mensal Estimado	Observação
Hospedagem da API	R\$ 0,00 – R\$ 80,00	Planos gratuitos disponíveis para volume baixo/médio
Domínio e SSL (opcional)	R\$ 5,00 – R\$ 40,00	Let's Encrypt fornece SSL gratuito
TOTAL ESTIMADO	R\$ 0,00 – R\$ 370,00/mês	Cenário 100% gratuito viável para empresa de pequeno porte

1.3.6.3 Benefícios e Retorno Esperado

Redução de custos com descarte: empresas gráficas de médio porte gastam, em média, R\$ 800,00 a R\$ 3.000,00 mensais com coleta e descarte de resíduos. Com o ReGraphik, estima-se redução de 30% a 60% nesse custo, resultando em economia estimada de R\$ 7.200,00 a R\$ 10.800,00 por ano.

Receita com venda de materiais recicláveis: papel/cartão (R\$ 0,20 a R\$ 0,60/kg) e vinil/PVC (R\$ 0,30 a R\$ 0,80/kg) podem gerar receita adicional estimada entre R\$ 300,00 e R\$ 1.600,00 por mês, a depender do volume.

Reaproveitamento interno (economia circular): o módulo de sugestões possibilita o uso de resíduos como matéria-prima para novos produtos, com redução de até 20% no custo de matéria-prima e melhoria na imagem da marca (alinhamento com ESG).

1.3.6.4 Análise Custo-Benefício

Categoria / Item	Tipo	Valor Anual Estimado
Desenvolvimento (amortizado em 3 anos)	Custo	R\$ 7.967,00/ano
Operação / Infraestrutura (cenário básico)	Custo	R\$ 0,00 – R\$ 1.200,00/ano
Manutenção e suporte técnico	Custo	R\$ 1.200,00 – R\$ 2.400,00/ano

Categoria / Item	Tipo	Valor Anual Estimado
Redução de custos com descarte	Benefício	R\$ 7.200,00 – R\$ 10.800,00/ano
Receita com venda de recicláveis	Benefício	R\$ 3.600,00 – R\$ 19.200,00/ano
Economia em matéria-prima (reaproveitamento)	Benefício	R\$ 2.400,00 – R\$ 6.000,00/ano
SALDO LÍQUIDO ESTIMADO	Lucro	R\$ 4.033,00 – R\$ 24.433,00/ano

A análise demonstra que o investimento no desenvolvimento do ReGraphik tem retorno positivo já no primeiro ano de operação, com payback estimado entre 3 e 8 meses, a depender do volume de resíduos gerados pela empresa. Além dos benefícios econômicos diretos, a adoção do sistema contribui para o cumprimento da Política Nacional de Resíduos Sólidos (Lei nº 12.305/2010).

1.3.7 RESULTADOS E CONCLUSÃO

Artefatos Técnicos Entregues

Artefato	Descrição	Status
Mini Mundo da Demanda	Contextualização do problema, atores e regras de negócio	Entregue
Modelo Conceitual (DER)	Diagrama ER com as 5 entidades do domínio (BRModelo)	Entregue
Modelo Lógico	Esquema relacional normalizado com PK, FK e restrições	Entregue
Modelo Físico	Script SQL completo com tabelas, índices e triggers	Entregue
Swagger / OpenAPI	Documentação interativa de todos os endpoints REST	Entregue

Artefato	Descrição	Status
Repositório GitHub	Código-fonte completo com histórico de commits	Entregue
Apresentação MVVM	Slides técnicos de arquitetura, padrões e integração	Entregue
Documentação TCC	Documentação completa do sistema (este documento)	Entregue

O desenvolvimento do ReGraphik ao longo do curso técnico em Desenvolvimento de Sistemas do SENAI proporcionou à equipe uma experiência abrangente de engenharia de software aplicada a um problema real. O projeto combinou conceitos de modelagem de dados, arquitetura de software, desenvolvimento de APIs REST, programação desktop e integração com serviços externos em nuvem.

Os principais aprendizados incluem: a aplicação prática do padrão arquitetural MVVM em uma aplicação WPF real; o design e implementação de uma API REST com ASP.NET Core, incluindo injeção de dependências e programação assíncrona; a modelagem relacional em três níveis (conceitual, lógico e físico); e a integração com serviços de nuvem como Firebase e Google Maps Places API.

O sistema cumpre os objetivos propostos ao oferecer uma solução concreta para o problema de gestão inadequada de resíduos no setor gráfico. O módulo de mapa, já funcional, demonstra a viabilidade técnica da integração entre o cliente desktop, a API REST e os serviços externos. Os demais módulos, em construção, têm arquitetura estabelecida e prontos para implementação.

Como trabalhos futuros, destacam-se: a implementação completa dos módulos de Dashboard, Cadastro de Resíduo, Sugestões e Relatórios; a adição de autenticação JWT na API REST; a ampliação dos testes unitários; e a eventual publicação da aplicação em ambiente de produção para uso real pela empresa AML.

O ReGraphik representa não apenas um exercício técnico, mas uma proposta de contribuição real à sustentabilidade no setor gráfico, demonstrando como o desenvolvimento de sistemas pode gerar valor econômico e ambiental concreto para as organizações.

Referências

MICROSOFT. ASP.NET Core documentation. Disponível em:
<https://learn.microsoft.com/aspnet/core>.

MICROSOFT. Windows Presentation Foundation. Disponível em:
<https://learn.microsoft.com/dotnet/desktop/wpf>.

GOOGLE. Firebase Realtime Database. Disponível em:
<https://firebase.google.com/docs/database>.

GOOGLE. Places API (New). Disponível em:
<https://developers.google.com/maps/documentation/places/web-service>.

AGAMBARD, T. et al. Leaflet.js. Disponível em: <https://leafletjs.com>.

FOWLER, Martin. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley, 2002.

MICROSOFT. Model-View-ViewModel (MVVM). Disponível em:
<https://learn.microsoft.com/dotnet/architecture/maui/mvvm>.

RICHARDSON, Leonard. RESTful Web Services. Sebastopol: O'Reilly Media, 2007.

SWAGGER. OpenAPI Specification. Disponível em: <https://swagger.io/specification>.

BRMODELO. Ferramenta de Modelagem Entidade-Relacionamento. Disponível em:
<https://www.sis4.com/brModelo>.

1.4 ANEXOS

1.4.1 BMG CANVAS

Parceiros Chave	Atividades Chave	Proposta de Valor	Rel. com Clientes	Segmento de Clientes
- Gráficas e editoras - Cooperativas de reciclagem - Fornecedores de tecnologia	- Desenvolvimento do software - Pesquisa de reaproveitamento - Apoio técnico	- Gestão de estoque reverso - Redução de descarte e custos - Economia circular	- Atendimento personalizado - Suporte técnico contínuo - Treinamentos	- Gráficas pequenas/médias - Empresas comunicação visual - ESG compliance
	Recursos Chave		Canais	
	- Equipe técnica - Banco de dados - Algoritmos de sugestão		- Plataforma digital - Consultores comerciais - Redes sociais	
Estrutura de Custos			Fluxo de Receitas	
- Desenvolvimento e hospedagem do sistema - Equipe de tecnologia e suporte - Marketing e divulgação			- Licenciamento SaaS (mensal/anual) - Consultoria Técnica - Suporte Premium e personalizações	

Figura — Business Model Canvas do ReGraphik.

1.4.2 SITUAÇÃO DE APRENDIZAGEM

A situação de aprendizagem que originou o desenvolvimento do ReGraphik está associada às unidades curriculares do Curso Técnico em Desenvolvimento de Sistemas do SENAI voltadas à modelagem de dados, à construção de aplicações desktop e à criação de APIs REST, articuladas ao eixo de Projetos Integradores. Nessa etapa, os estudantes foram desafiados a identificar uma demanda real de mercado ou de uma organização parceira e a propor uma solução de software completa, contemplando desde o levantamento de requisitos até a documentação técnica de entrega.

A partir do contato com a empresa AML, do setor gráfico, a equipe identificou a ausência de controle estruturado sobre os resíduos sólidos gerados no processo produtivo — papel, cartão, vinil, lona e PVC —, descartados sem qualquer critério de reaproveitamento. Essa situação-problema tornou-se o ponto de partida da situação de aprendizagem: aplicar os conhecimentos técnicos do curso à construção de uma solução de gestão de estoque reverso capaz de reduzir o desperdício e agregar valor econômico e ambiental à empresa.

Ao longo do desenvolvimento, os componentes curriculares de Modelagem de Banco de Dados, Programação Orientada a Objetos, Desenvolvimento de Aplicações Desktop, Desenvolvimento Web/API e Engenharia de Software foram mobilizados de forma integrada: a modelagem conceitual, lógica e física do banco de dados; a implementação do padrão arquitetural MVVM no cliente WPF; a construção de uma API REST documentada em ASP.NET Core; e a

integração com serviços externos em nuvem (Firebase) e de geolocalização (Google Maps Places API).

Dessa forma, a situação de aprendizagem cumpriu seu papel formativo ao exigir da equipe a articulação entre teoria e prática, o trabalho colaborativo em equipe multidisciplinar e a resolução de um problema real e relevante para o setor produtivo, consolidando as competências técnicas previstas na matriz curricular do curso.

Repositório: <https://github.com/BrunoMaiaSenai/ReGraphikApp>